



Documento Mestre do Sistema SIGMA

Arquitetura, integrações, infraestrutura e fluxos operacionais

Sumário

1. Sumário executivo	3
2. Visão funcional, escopo e objetivos	3
2.1 Áreas atendidas	3
2.2 Objetivos consolidados	4
2.3 Organização funcional	4
2.4 Módulos do sistema	5
2.5 Fora do escopo desta Versão	6
3. Arquitetura técnica e infraestrutura	6
3.1 Backend	6
3.2 Frontend	6
3.3 Rede dos equipamentos	7
3.4 Infraestrutura	7
3.4 Hospedagem e execução em produção	7
3.5 Arquitetura lógica	10
3.6 Aplicativo Android	11
4. Dados, bancos e modelo local	15
4.1 PostgreSQL local	15
4.2 Oracle ERP	16
4.3 Oracle Alchemy	17
4.4 Campos e tabelas customizadas no Oracle ERP	17
4.5 Índices personalizados	20
4.6 Principais modelos locais do projeto	23
5. Navegação e rotas principais	28
5.1 Raiz	28
5.2 Telemetria	29
5.3 Produção	29
5.4 Qualidade	29
5.5 PCP	30
5.6 Manutenção	30
5.7 WebSocket	31
6. Integrações externas e contratos de comunicação	31
6.1 ERP Oracle - consulta direta	31
6.2 ERP Sapiens/Senior - SOAP	32
6.3 WMS XC API	37
6.4 WMS via DBLINK Oracle	38
6.5 Oracle Alchemy	38
6.6 MQTT	39
6.7 LDAP / Active Directory	39
6.8 SMTP	39
6.9 Retornos, status e tratamento de erro das integrações	40
7. Operação em background, filas e monitoramento	40
7.1 Inicialização	41
7.2 Supervisor	41
7.3 Tabela de workers	41
7.3.1 Coleta de Telemetria	43
7.3.2 Escalabilidade das consultas de histórico	44
7.4 Timeout e reprocessamento	45
7.5 Manual operacional do painel de Status (Services)	45
8. Fluxos de negócio	47
8.1 Sequenciamento de produção	47
8.1.1 Calendário de OPs	48
8.2 Apontamentos de produção	49
8.3 Tempos de produção, paradas e justificativas	51
8.4 Apontamento de componente	54
8.5 Estorno de componente	55
8.6 Liberação de lotes	56
8.7 Área vermelha	57

8.8 Consulta de lote, etiqueta e rastreamento	58
8.9 Status de recursos	59
8.10 Manutenção	59
9. Parametrizações críticas	60
10. Segurança e acessos	61
10.1 Autenticação	61
10.2 Autorização	61
10.3 Observações de segurança	61

Versão: Estado atual Data de atualização: 2026-08-12 Base documental: Projeto SIGMA e ambiente de produção nex01

Autor: Silonei Duarte

1. Sumário executivo

O SIGMA conecta a operação industrial diária com os sistemas corporativos que registram produção, estoque, qualidade e manutenção. Na prática, ele funciona como uma camada operacional entre o usuário de fábrica e os sistemas como ERP Senior/Sapiens, WMS, Alchemy, MQTT de equipamentos e banco local PostgreSQL.

O objetivo do sistema não é apenas “mostrar telas”. Ele organiza fluxos que normalmente ficariam espalhados entre apontamentos manuais, consultas ao ERP, decisões de qualidade, controles de lote, movimentações de estoque, chamados de manutenção e acompanhamento de recursos. Por isso, o SIGMA possui duas naturezas ao mesmo tempo: é um portal operacional para usuários e também um integrador técnico entre sistemas.

Quando um operador aponta uma OP, quando a qualidade libera um lote, quando um componente é consumido, quando uma pendência precisa ir ao WMS ou quando uma balança envia leitura em tempo real, o SIGMA registra, valida, envia, interpreta retorno e mantém rastreabilidade. Essa rastreabilidade é importante porque a operação industrial não pode depender de uma chamada externa perfeita o tempo todo. Se ERP ou WMS falham, o sistema precisa deixar claro o que ficou pendente, o que foi integrado e o que precisa de reprocessamento.

O sistema tem três papéis principais:

1. Interface operacional para usuários de produção, qualidade, manutenção e administração.
2. Camada de integração entre dados locais, ERP Senior/Sapiens, WMS, Oracle Alchemy e dispositivos MQTT.
3. Orquestrador de filas e serviços em background, com controle de status, timeout e reprocessamento.

Em linguagem simples: o SIGMA recebe a ação do usuário, consulta o que precisa nos bancos externos, grava uma trilha local no PostgreSQL, chama ERP ou WMS quando necessário e depois mostra se a ação foi concluída ou se ficou pendente.

2. Visão funcional, escopo e objetivos

O escopo do SIGMA cobre os processos que precisam unir operação de fábrica, rastreabilidade local e integração com sistemas externos. Ele não substitui o ERP nem o WMS. O papel dele é facilitar a execução operacional, reduzir retrabalho e registrar uma trilha própria do que foi feito, enviado, aceito ou recusado.

O principal objetivo de negócio é reduzir a distância entre o que acontece na fábrica e o que precisa ficar registrado nos sistemas oficiais. Em vez de depender de planilhas, consultas manuais, chamadas diretas ao ERP ou controles paralelos, o sistema concentra os fluxos em uma interface única e registra a trilha operacional.

Outro objetivo central é dar visibilidade ao erro. Uma integração pode falhar por indisponibilidade de ERP, retorno inválido, timeout, dado incorreto ou regra de negócio recusada. O SIGMA precisa deixar a pendência visível e reprocessável, em vez de simplesmente perder a ação ou obrigar o usuário a repetir tudo manualmente.

Tecnicamente, o SIGMA separa controle local de efetivação externa. O PostgreSQL local guarda a operação do SIGMA: filas, logs, parâmetros, cadastros, reuniões, chamados e paradas de máquina. O banco Oracle ERP, Alchemy e XC WMS são consultados para buscar informações oficiais ou complementares. As alterações no ERP são feitas quase que unicamente por webservices Sapiens/Senior, respeitando as regras do ERP.

2.1 Áreas atendidas

Na parte de cadastro e parametrização, o sistema controla a estrutura usada pelos demais fluxos: empresas, filiais, departamentos, setores, centros de recurso, recursos, turnos, calendários, horas extras e taras. Esses cadastros não são apenas informativos. Eles definem como cada recurso trabalha, quais depósitos devem ser usados, quais parâmetros se aplicam por filial ou centro e como o OEE deve calcular o planejado.

Na autenticação, o SIGMA usa login Django e integração LDAP/Active Directory. Isso permite que o acesso acompanhe o ambiente corporativo. O usuário pode ser criado ou atualizado automaticamente a partir do AD, mas continua tendo dados

loais necessários para operação, como filial, operador ERP e página inicial.

Na produção, o sistema cobre sequenciamento de OPs, apontamento de OP, apontamento de componentes, estorno de componentes, logs de integração, status de recursos e controle de OP ativa por recurso. Essa área transforma a ação da fábrica em registro local e, quando aplicável, em chamada ao ERP Senior/Sapiens.

Na qualidade, o sistema cobre liberação de lotes, área vermelha, consulta de lotes, rastreamento, etiquetas, observações de etiqueta e integração WMS. A decisão de qualidade pode gerar efeitos diferentes: liberar lote, mandar para área vermelha, refugar, reclassificar, transferir produto ou criar pendência WMS.

Na manutenção, o sistema controla chamados, QR Code por recurso, ordens de serviço, interações e notificações por e-mail. O objetivo é vincular problemas e atendimentos aos recursos produtivos, criando histórico de manutenção e comunicação entre solicitantes, responsáveis e observadores.

No OEE e status de recursos, o SIGMA recebe mensagens MQTT dos equipamentos para atualizar balança em tempo real e consolida planejado diário com base em calendário, turnos e horas extras. As paradas de máquina são registradas em tabela própria, permitindo identificar se existe parada em aberto por recurso.

Em questão a esses dois últimos, embora, manutenção e OEE estejam desenvolvidos na maior parte, atualmente não estão em uso produtivo, porem foram mantidos visando futuras integrações.

Nas integrações, o sistema conversa com PostgreSQL local, Banco Oracle ERP Senior, webservices Sapiens G5, WMS XC API, WMS via DBLINK Oracle, Banco Oracle Alchemy, MQTT, WebSocket e SMTP Microsoft 365. Cada integração tem uma responsabilidade própria: algumas consultam dados, outras efetivam movimentos e outras servem para notificação ou tempo real.

2.2 Objetivos consolidados

Os objetivos de negócio são:

- Reduzir operação manual entre produção, qualidade, ERP e WMS.
- Garantir rastreabilidade local das integrações enviadas ao ERP e WMS.
- Apoiar decisões de qualidade sobre liberação, refugo e reclassificação de lotes.
- Concentrar status de recursos e apontamentos em uma interface única.
- Permitir reprocessamento controlado de pendências quando uma integração falha.
- Integrar operação de manutenção ao cadastro de recursos.

Os objetivos técnicos são:

- Manter uma base local PostgreSQL como controle operacional e fila de integração.
- Consultar Oracle ERP e Oracle Alchemy sem copiar dados desnecessários.
- Efetivar movimentos no ERP via webservices oficiais Sapiens/Senior.
- Enviar eventos de WMS via API HTTP JSON.
- Executar workers automáticos com monitoramento de timeout.
- Evitar duplicidade de workers usando trava no PostgreSQL.
- Expor telas em tempo real via WebSocket quando necessário.

2.3 Organização funcional

A organização funcional abaixo não representa cronograma nem ordem de desenvolvimento. Ela serve para decompor o SIGMA em blocos compreensíveis. A ideia é responder: “quais grandes partes formam o sistema?” e “que entregas existem dentro de cada parte?”.

Para leitores não técnicos, esta divisão deve ser lida como um mapa de responsabilidades. Por exemplo, “Produção” concentra tudo que envolve OP, sequenciamento, apontamento e componentes. “Qualidade” concentra liberação de lote, área vermelha, rastreamento e WMS. “Integrações externas” agrupa os pontos em que o SIGMA deixa de trabalhar apenas com dados locais e passa a consultar ou enviar informações para outros sistemas.

Essa divisão também ajuda a entender impacto de mudanças. Uma alteração em “Parâmetros de filial”, por exemplo, pode afetar Produção e Qualidade, porque os mesmos parâmetros definem depósitos, transações e locais usados nos fluxos de integração.

- **1.1 Aplicação web:** estrutura da aplicação web, rotas, telas, renderização das páginas e entrada ASGI usada pelo servidor.
- **1.2 Autenticação:** login de usuários, integração com LDAP/Active Directory, grupos, permissões e controle de acesso às telas.
- **2.1 Estrutura empresarial:** cadastro e organização de empresa, filial, departamento e setor, usados como base para

permissões, filtros e parâmetros.

- **2.2 Estrutura produtiva:** cadastro de centros de recurso, recursos produtivos e parâmetros operacionais usados por produção, OEE, manutenção e integrações.
- **2.3 Calendário e turnos:** definição de calendário, eventos, turnos base, turnos por recurso e horas extras para cálculo de disponibilidade e planejado.
- **3.1 Sequenciamento:** consulta de OPs no ERP, organização da sequência por recurso e manutenção da ordem operacional local.
- **3.2 Apontamento OP:** tela de apontamento da produção, geração de registros locais, envio ao ERP por webservice e acompanhamento dos retornos. Inclui a versão Multi-OP (View 3) para recursos que acoplam várias OPs ao mesmo tempo mas apontam uma por vez, com consumo de bobina de matéria-prima rateado entre elas.
- **3.3 Apontamento componente:** registro do consumo de componentes, validação do contexto ERP/WMS, envio ao ERP e ajuste de estoque quando aplicável. Inclui a baixa de bobina de matéria-prima gerada pela View 3, enviada por webservice próprio (TratarBaixa).
- **3.4 Estorno componente:** controle local de estornos de componentes, envio ao ERP e rastreabilidade do retorno da integração.
- **4.1 Liberação de lote:** consulta de dados no ERP e Alchemy, decisão de qualidade, criação da liberação local e envio ao WMS quando houver novo lote.
- **4.2 Área vermelha:** controle de reunião, definição de destino do lote, comunicação com ERP/WMS e uso de dados de qualidade do Alchemy.
- **4.3 Consulta e etiqueta:** consulta de lotes e saldos no ERP, visualização do resultado e emissão de etiqueta em PDF/HTML.
- **4.4 Rastreamento:** acompanhamento da cadeia do lote, movimentos no ERP e informações complementares de qualidade vindas do Alchemy. A tela permite informar empresa e lote diretamente; também é aberta pelos links de Consulta de Lotes e Área Vermelha quando o lote já está definido. Antes da consulta pesada, abre imediatamente com o estado *Rastreando lote...*
- **5.1 Chamados:** abertura e acompanhamento de chamados por recurso, QR Code, responsáveis, interações e notificações por e-mail.
- **5.2 Ordens de serviço:** controle de ordens de serviço, responsáveis, interações, histórico operacional e comunicação por e-mail.
- **6.1 MQTT:** recebimento de leituras dos recursos, comunicação de balança e atualização em tempo real.
- **6.2 Telemetria HTTP:** coleta contínua de valores posicionais em páginas HTML de recursos, interpretação tipada, armazenamento de leituras e avaliação automática de parada por sinal.
- **6.3 OEE planejado:** consolidação do planejado diário, uso de calendário/turnos e reprocessamento quando dados operacionais mudam.
- **7.1 ERP Oracle:** consultas diretas ao Oracle ERP para buscar OP, lote, saldo, movimentos, operadores e demais informações oficiais.
- **7.2 ERP Sapiens SOAP:** envio de apontamentos, componentes, estornos e movimentações para efetivação pelas regras do ERP Senior/Sapiens.
- **7.3 WMS API:** envio de novo lote, ajuste de estoque e pendências de integração relacionadas ao WMS.
- **7.4 Alchemy:** consulta de análises de bobina e dados complementares usados nos fluxos de qualidade e rastreamento.
- **8.1 Hospedagem:** execução em Ubuntu Server com systemd, Daphne, Nginx, PostgreSQL local e variáveis de ambiente.
- **9.1 Workers:** execução dos serviços em background, controle de filas, timeout, status operacional e reprocessamento.

2.4 Módulos do sistema

O SIGMA possui uma base de configuração que sustenta todos os módulos: bancos de dados, integrações externas, autenticação, e-mail, arquivos estáticos, fuso horário, idioma, rotas HTTP e rotas WebSocket. Essa base define como a aplicação se conecta aos sistemas externos, como publica suas telas e quais valores vêm do ambiente de execução em vez de ficarem fixos no código.

Cadastros, usuários e estrutura fabril sustentam todos os outros módulos. Eles definem quem usa o sistema, a qual filial o usuário pertence, quais empresas existem, como a fábrica está organizada e quais recursos produtivos podem receber apontamentos, chamados, turnos e sinais MQTT.

A estrutura principal segue esta hierarquia:

Empresa -> Filial -> Departamento -> Setor -> Centro de Recurso -> Recurso

Os parâmetros de integração também seguem uma regra de herança. Primeiro o sistema usa o parâmetro da filial. Se houver configuração no centro de recurso, ela substitui a configuração da filial. Em alguns casos, o recurso possui parâmetros próprios, que ficam acima dos demais. Essa hierarquia evita repetir configurações e permite exceções por máquina ou centro.

Autenticação, permissões e segurança de acesso controlam quem entra e o que cada usuário pode fazer. A autenticação principal usa LDAP/Active Directory, permitindo que usuários corporativos entrem com suas credenciais de rede. Depois do login, o acesso às telas é controlado por permissões Django e por regras de staff/superusuário.

O módulo de produção cobre a rotina operacional da fábrica. Ele consulta OPs no ERP, permite organizar sequenciamento local, registra apontamentos, controla OP ativa por recurso, envia apontamentos ao ERP e mantém logs locais para reproprocessamento. O ponto mais importante desse módulo é a fila local: um apontamento não depende de o ERP responder imediatamente para existir no SIGMA.

O módulo de qualidade controla a destinação dos lotes após produção. Ele consulta saldos e situação no ERP, consulta dados de análise no Alchemy, localização do lote no WMS ou nos Parâmetros, permite liberar lotes, conduzir reuniões de área vermelha, registrar refugo/reclassificação, gerar etiquetas, rastrear lotes e enviar pendências ao WMS.

O módulo de manutenção registra problemas e atividades de manutenção ligadas aos recursos da fábrica. Chamados podem ser abertos pela tela ou por QR Code do recurso. Ordens de serviço organizam execução, responsáveis, previsão, status e histórico de interações.

OEE, MQTT, telemetria HTTP e paradas de máquina acompanham sinais dos recursos. O SIGMA recebe mensagens MQTT para atualizar balança em tempo real via WebSocket, coleta valores HTTP configurados por recurso, avalia regras de parada por sinal, registra paradas em tabela própria e recalcula planejado OEE a partir de turnos, calendários e horas extras.

2.5 Fora do escopo desta Versão

Este documento ainda não substitui uma validação formal com usuários-chave. Ele descreve o sistema a partir do projeto, das configurações e do ambiente de produção, mas algumas regras devem ser alinhadas com as áreas de negócio para garantir que o comportamento documentado corresponde ao processo aprovado.

Também não faz parte desta Versão um desenho BPMN formal, um manual operacional tela a tela, um inventário completo de permissões por grupo real no banco de produção, uma auditoria de segurança ou uma revisão de performance baseada em métricas históricas. Esses itens podem ser produzidos depois, usando este documento mestre como base.

3. Arquitetura técnica e infraestrutura

O SIGMA é uma aplicação web server-side. Isso significa que a maior parte das regras de negócio roda no servidor, dentro do Django, e o navegador atua como interface para o usuário. Algumas telas usam JavaScript para melhorar a operação, mas a decisão de salvar, integrar, validar permissão e chamar sistemas externos fica no backend.

As tecnologias foram escolhidas para atender três necessidades principais:

1. Manter uma aplicação web com usuários, permissões, telas e formulários.
2. Conversar com bancos e sistemas industriais diferentes, como PostgreSQL, Oracle ERP, Alchemy, WMS e MQTT.
3. Executar tarefas automáticas em segundo plano, como envio de filas, importação de paletes e consolidação de OEE.

3.1 Backend

O backend é escrito em Python e usa Django como estrutura principal. O Django organiza as telas, rotas, modelos de banco, autenticação, permissões e formulários. A aplicação roda em ASGI por meio do Daphne, permitindo HTTP normal e WebSocket no mesmo processo.

O PostgreSQL é o banco operacional local. Ele guarda usuários, cadastros, filas, logs, parâmetros e histórico operacional. O acesso normal da aplicação a esse banco é feito pelo ORM do Django, ou seja, os modelos Python representam as tabelas locais e concentram as operações de gravação, consulta e atualização. O Oracle não substitui o PostgreSQL; ele entra como sistema externo consultado para OPs, estoque, lote, movimentos e operadores.

As chamadas HTTP externas usam `requests` nas chamadas SOAP ao Sapiens e nas chamadas JSON ao WMS. Para MQTT, a aplicação usa `paho-mqtt`, mantendo um cliente conectado ao broker para receber leitura de balança.

Para documentos e saídas auxiliares, o sistema usa bibliotecas como `pandas`, `reportlab` e `qrcode`, permitindo exportação de Excel, geração de PDF/etiquetas e QR Codes.

3.2 Frontend

O frontend é composto por templates renderizados pelo Django. Isso quer dizer que as páginas são montadas no servidor e entregues prontas ao navegador. O estilo visual usa Tailwind/theme local e alguns elementos com Bootstrap Icons.

O JavaScript é usado principalmente onde há interação operacional mais dinâmica: sequenciamento por arrastar/organizar, atualização de balança em tempo real, troca de OP e filtros de tela. O WebSocket permite que certas informações cheguem ao navegador sem recarregar a página.

O cabeçalho possui seletor de tema claro/escuro. A escolha é salva no navegador e aplicada novamente ao abrir outra tela. O tema escuro ajusta fundos, textos, bordas, campos, tabelas, modais, links e estados de seleção para manter contraste; as telas que não usam o template base, como Rastreamento de Lote, possuem o mesmo seletor e tratamento visual próprio.

3.3 Rede dos equipamentos

Os equipamentos precisam de acesso HTTPS ao SIGMA para abrir a aplicação e carregar todos os estilos, ícones e recursos do calendário. Esses arquivos são distribuídos localmente pelo próprio servidor, sem dependência de CDN ou de acesso externo.

Endereço	Finalidade
<code>sigma.indaialpapel.com.br</code>	Aplicação SIGMA e seus arquivos estáticos locais, incluindo o CSS principal.

Quando os equipamentos tiverem bloqueio de acesso externo, não é necessária nenhuma liberação para bibliotecas visuais. O acesso a `sigma.indaialpapel.com.br` é suficiente para a interface completa.

3.4 Infraestrutura

Em produção, a aplicação roda em um servidor Ubuntu. O systemd mantém o serviço ativo e reinicia em caso de queda. O Daphne atende a aplicação na porta interna 8000. O Nginx atua como camada web/reverse proxy.

O banco PostgreSQL roda no mesmo servidor, o que reduz latência para as filas e cadastros locais. As conexões com Oracle, WMS, MQTT, LDAP e Sapiens dependem da rede interna.

Resumo do ambiente:

Camada	Tecnologia/valor
Sistema operacional	Ubuntu 24.04.4 LTS
Host	nex01
Usuário do processo	nexus
Diretório da aplicação	/opt/SIGMA
Processo web	Daphne ASGI
Porta interna	8000
Banco local	PostgreSQL 18.4 com TimescaleDB 2.29.1
Arquivo de ambiente	/etc/sigma/sigma.env

3.4 Hospedagem e execução em produção

3.4.1 Servidor de produção

Esta seção descreve onde a aplicação roda de fato. O objetivo não é documentar código, mas permitir que alguém de infraestrutura ou suporte saiba onde procurar quando a aplicação estiver fora do ar, lenta ou com erro de ambiente.

O serviço principal se chama `sigma.service`. Ele roda com o usuário Linux `nexus` e usa o diretório `/opt/SIGMA` como raiz da aplicação. Quando o servidor reinicia, o systemd sobe esse serviço automaticamente. Se o processo cair, a diretiva `Restart=always` tenta reiniciar.

Dados do ambiente:

Item	Valor
Usuário SSH	nexus
Host	nex01

Item	Valor
Sistema operacional	Ubuntu 24.04.4 LTS
Diretório inicial	/home/nexus
Diretório da aplicação	/opt/SIGMA
Serviço systemd	sigma.service
Processo web	Daphne
Porta interna	8000
Banco local	PostgreSQL 18.4 com TimescaleDB 2.29.1

3.4.2 Unidade systemd

O systemd é o gerenciador responsável por manter a aplicação rodando. Ele define quem executa o processo, de onde ele executa e qual comando inicia o servidor web Python. A aplicação não é iniciada manualmente com `python manage.py runserver` em produção; ela é iniciada pelo Daphne por meio do systemd.

Serviço:

```
[Unit]
Description=SIGMA Django
After=network.target

[Service]
User=nexus
WorkingDirectory=/opt/SIGMA
Environment="PYTHONUNBUFFERED=1"
ExecStart=/opt/SIGMA/.venv/bin/python -u /opt/SIGMA/.venv/bin/daphne -b 127.0.0.1 -p 8000 SIGMA.asgi:application
Restart=always

[Install]
WantedBy=multi-user.target
```

Drop-in do systemd:

O drop-in é um complemento da configuração principal do serviço. Ele permite adicionar ou sobrescrever partes do `sigma.service` sem editar diretamente o arquivo base da unit. No SIGMA, esse complemento é usado para informar ao systemd que o serviço deve carregar as variáveis de ambiente do arquivo `/etc/sigma/sigma.env` antes de iniciar a aplicação. É por esse caminho que o processo recebe configurações como bancos, integrações, chaves de ambiente, e-mail e demais parâmetros externos ao código.

```
[Service]
EnvironmentFile=/etc/sigma/sigma.env
```

3.4.3 Processo em produção

O processo web usa a entrada ASGI (`SIGMA.asgi:application`). ASGI é a interface entre o servidor web Python e a aplicação Django para aplicações assíncronas. Na prática, é o ponto por onde o Daphne chama o SIGMA para atender requisições. Isso é importante porque ASGI permite tanto requisições HTTP comuns quanto WebSocket, necessário para recursos como balança em tempo real e atualização de OP.

Comando do processo web:

```
/opt/SIGMA/.venv/bin/python -u /opt/SIGMA/.venv/bin/daphne -b 127.0.0.1 -p 8000 SIGMA.asgi:application
```

Serviços ativos relacionados:

- `sigma.service`: ativo e rodando.

- `nginx.service`: ativo e rodando.
- `postgresql@18-main.service`: ativo e rodando.

Comandos operacionais de atualização:

```
cd /opt/SIGMA
.venv/bin/python manage.py migrate
.venv/bin/python manage.py tailwind build
.venv/bin/python manage.py collectstatic --noinput
sudo systemctl reset-failed sigma
sudo systemctl restart sigma
systemctl status sigma --no-pager -l
```

3.4.3.1 Atualização de PostgreSQL e TimescaleDB

Os pacotes `postgresql-18` e `postgresql-18-timescaledb` permanecem em *hold* no APT. Portanto, um `apt upgrade` comum pode atualizar os demais pacotes do servidor, mas não altera o PostgreSQL nem o TimescaleDB. Não remover o *hold* nem atualizar esses pacotes fora de janela de manutenção aprovada.

O TimescaleDB possui duas versões que precisam permanecer alinhadas: a biblioteca instalada no servidor e a extensão SQL registrada em cada banco. Depois de instalar uma versão nova do pacote, executar imediatamente a atualização da extensão no banco `sigma`. Se isso não ocorrer, a aplicação não consegue abrir conexões com o PostgreSQL.

Procedimento para uma atualização planejada do TimescaleDB:

```
# 1. Conferir os holds e criar dump lógico antes da alteração.
apt-mark showhold
sudo -u postgres pg_dump -Fc -d sigma \
  -f /var/backups/sigma/sigma_pre_timescaledb_$(date -u +%Y%m%dT%H%M%SZ).dump

# 2. Liberar somente os pacotes envolvidos e instalar a versão aprovada.
sudo apt-mark unhold postgresql-18 postgresql-18-timescaledb
sudo apt-get update
sudo apt-get install postgresql-18-timescaledb=<VERSAO_APROVADA>

# 3. Este DEVE ser o primeiro comando da sessão psql no banco sigma.
sudo -u postgres psql -X -d sigma -v ON_ERROR_STOP=1 \
  -c "ALTER EXTENSION timescaledb UPDATE;"

# 4. Reiniciar os consumidores de conexões e validar as versões.
sudo systemctl restart pgbouncer sigma
sudo -u postgres psql -X -d sigma -P pager=off \
  -c "SELECT extname, extversion FROM pg_extension WHERE extname = 'timescaledb';"
sudo -u postgres psql -X -d sigma -P pager=off \
  -c "SELECT value FROM _timescaledb_catalog.metadata WHERE key = 'timescaledb_version';"
cd /opt/SIGMA && .venv/bin/python manage.py shell -c "from django.db import connection; connection.ensure_connection();"

# 5. Reter novamente os pacotes após a validação.
sudo apt-mark hold postgresql-18 postgresql-18-timescaledb
```

Usar `psql -X` evita comandos automáticos do perfil do cliente. O `ALTER EXTENSION` deve ser o primeiro comando da sessão porque o TimescaleDB não aceita carregar uma biblioteca nova com o catálogo SQL ainda na versão anterior. Não executar `apt update && apt upgrade -y` em produção como forma de atualizar PostgreSQL ou TimescaleDB.

O caminho da aplicação em produção é sensível a maiúsculas e minúsculas. O diretório correto é `/opt/SIGMA`; caminhos como `/opt/sigma` ou `/opt/Sigma` não existem nesse ambiente.

Quando `uv` estiver disponível no `PATH` do usuário `nexus`, os comandos Python também podem ser executados com `uv run python`. Se o shell ainda não reconhecer `uv`, usar `.venv/bin/python` evita depender do `PATH`.

3.4.4 Variáveis de ambiente de produção

As variáveis de ambiente são a camada de configuração sensível do sistema. Elas indicam para onde o SIGMA deve conectar, quais bancos usar, qual URL de webservice chamar e quais credenciais utilizar. As credenciais, chaves e usuários não aparecem neste documento funcional, este documento registra apenas valores não sensíveis.

Arquivo usado:

```
/etc/sigma/sigma.env
```

Valores não sensíveis:

Variável	Valor
DJANGO_DEBUG	False
DJANGO_ALLOWED_HOSTS	sigma.indaialpapel.com.br
DJANGO_HTTPS_ENABLED	True
DB_DEFAULT_NAME	sigma
DB_DEFAULT_HOST	127.0.0.1 (PgBouncer)
DB_DEFAULT_PORT	6432
POSTGRES_USA_PGBOUNCER	1
POSTGRES_DIRECT_HOST	127.0.0.1
POSTGRES_DIRECT_PORT	5432
ORACLE_ERP_NAME	172.16.30.51/dbdev
ORACLE_ALCHEMY_NAME	172.16.30.10/dbprod
SAPIENS_URL_BASE	http://172.16.30.26:8080
WMS_XC_API_URL	http://10.61.31.10:1919/
MQTT_HOST	127.0.0.1
MQTT_PORT	1883
LDAP_SERVER_URI	ldaps://DC01.indaialpapel.com.br:636
LDAP_USER_DOMAIN	indaialpapel.com.br
LDAP_CA_CERT_FILE	/opt/SIGMA/certs/ca_indaialpapel.pem
PORTAL_BASE_URL	https://sigma.indaialpapel.com.br
NPM_BIN_PATH	/usr/local/bin/npm

3.5 Arquitetura lógica

```

Usuários
|
| HTTP / WebSocket
v
SIGMA / Django / Daphne / Channels
|
+--> PostgreSQL local: dados operacionais, filas, parâmetros, status
|
+--> Oracle ERP: consultas ao ERP
|
+--> Sapiens SOAP: efetivação de apontamentos e movimentos
|
+--> WMS XC API: envio de novo lote e ajuste de estoque

```

```

|
+--> WMS via DBLINK no Oracle: consulta de paletes/local/saldo
|
+--> Oracle Alchemy: análises de bobina
|
+--> MQTT: leitura de recursos e balança
|
+--> SMTP: notificações de manutenção

```

3.6 Aplicativo Android

O aplicativo Android interno fica em `mobile/` e usa Capacitor 8.4.2. Ele funciona como contêiner do SIGMA web e carrega somente <https://sigma.indaialpapel.com.br>; tráfego HTTP em texto claro permanece desativado e os logs do Capacitor ficam desabilitados. Como as telas são entregues pelo Django, mudanças nas páginas e regras do SIGMA entram em vigor sem gerar outro APK. Uma nova compilação é necessária quando mudar a configuração ou a camada nativa do aplicativo. A versão atual do APK é 1.0.4 (versionCode 5).

Configuração atual:

Item	Valor
App ID	<code>br.com.indaialpapel.sigma</code>
Nome	SIGMA
Ícone	Logo IPEL de <code>static/images/logo.png</code>
Capacitor	8.4.2
Android mínimo	API 24 / Android 7
Android alvo	API 36
URL	<code>https://sigma.indaialpapel.com.br</code>
Cleartext	Desativado
Backup do aplicativo	Desativado
Versão	1.0.4 (versionCode 5)
Permissões Android	Internet, câmera, localização aproximada/exata, notificações e leitura de imagens

O dispositivo precisa resolver o domínio pelo DNS interno e acessar a faixa corporativa 172.16.0.0/16, diretamente ou pela VPN corporativa.

As permissões sensíveis não são solicitadas ao abrir o aplicativo. O Android apresenta a confirmação somente quando uma função do SIGMA tentar usar câmera, localização, notificações ou leitura direta de imagens. A câmera é um recurso opcional e não restringe a instalação em dispositivos sem esse hardware. Galeria e arquivos também podem usar o seletor nativo do Android, sem conceder acesso amplo ao armazenamento. A declaração no manifesto apenas prepara o aplicativo; cada recurso ainda precisa ser implementado na página ou na camada nativa que irá utilizá-lo.

Os arquivos gerados pelo SIGMA, como os PDFs de QR Code da manutenção, são tratados pela camada nativa. O aplicativo repassa ao gerenciador de downloads do Android a URL HTTPS, o cookie da sessão autenticada e o agente do navegador. O arquivo recebe um horário no nome para evitar colisões, fica na pasta pública **Downloads** e o Android informa quando o download termina. Links que usam nova aba no navegador são abertos no próprio WebView quando executados pelo aplicativo, permitindo que esse tratamento seja acionado.

3.6.1 Modelo do aplicativo e mapa de arquivos

O APK não contém uma cópia das telas do SIGMA. O Capacitor abre a aplicação web de produção definida em `mobile/capacitor.config.ts`; por isso, mudanças em templates, views e regras Django são publicadas no servidor e entram no aplicativo sem recompilar o APK. Uma nova versão do APK é necessária quando mudar configuração nativa, permissões, plugins, ícone, App ID, versão Android ou projeto Firebase do aplicativo.

A pasta `mobile/` pertence ao código-fonte usado para gerar o APK e não precisa existir em `/opt/SIGMA` no servidor de produção. O servidor executa o backend Django, guarda a credencial de envio em `/etc/sigma/firebase-admin.json` e mantém o APK disponível para usuários autenticados em `/opt/SIGMA/artifacts/SIGMA.apk`. A estrutura equivalente em desenvolvimento é `artifacts/SIGMA.apk`.

Arquivo	Responsabilidade
<code>mobile/capacitor.config.ts</code>	App ID, nome, URL HTTPS, logs e comportamento dos plugins
<code>mobile/package.json</code>	Capacitor 8.4.2, Push Notifications 8.1.2 e comandos de build
<code>mobile/android/build.gradle</code>	Android Gradle Plugin e Google Services 4.5.0
<code>mobile/android/app/build.gradle</code>	versão, APIs Android e assinatura release
<code>mobile/android/app/src/main/AndroidManifest.xml</code>	permissões e requisitos de hardware
<code>mobile/android/app/google-services.json</code>	identifica o aplicativo Android no projeto Firebase
<code>mobile/build-release.ps1</code>	sincroniza o Capacitor, assina e atualiza <code>mobile/dist/SIGMA.apk</code> e <code>artifacts/SIGMA.apk</code>
<code>mobile/keystore/sigma-release.jks</code>	chave permanente usada para assinar todas as versões release
<code>artifacts/SIGMA.apk</code>	arquivo entregue pela view autenticada de notificações; ignorado pelo Git

`mobile/android/app/google-services.json`, a chave Firebase do servidor, o keystore, suas senhas e o APK gerado ficam fora do Git. O `google-services.json` identifica o aplicativo, mas não autoriza o envio de mensagens. A autorização de envio pertence exclusivamente à conta de serviço instalada no servidor.

3.6.2 Arquitetura das notificações

O projeto Firebase é `sigma-f611c` e o aplicativo Android registrado usa o pacote `br.com.indaialpapel.sigma`. O cliente usa `@capacitor/push-notifications`; o servidor usa `firebase-admin 7.5.0`.

Aplicativo Android autenticado

- > solicita permissão somente após ação do usuário
- > Google Play Services gera o token FCM do aparelho
- > POST autenticado e protegido por CSRF registra o token no Django
- > tabela `dispositivos_notificacao` associa token e usuário

Regra de negócio no Django

- > `enviar_notificacao_usuario(usuario, título, mensagem, dados)`
- > Firebase Admin autentica com a conta de serviço do servidor
- > FCM entrega a mensagem ao token ativo do usuário
- > Android exibe a notificação no canal `sigma_geral`

Componentes do backend:

Arquivo	Responsabilidade
accounts/models/notificacoes.py	modelo DispositivoNotificacao
accounts/migrations/0018_dispositivonotificacao.py	criação da tabela dispositivos_notificacao
accounts/views/notificacoes.py	página, download autenticado do APK, registro do token e envio do teste
accounts/services/notificacoes.py	inicialização do Firebase Admin e função central de envio
templates/accounts/notificacoes_teste.html	autorização, registro e teste no Android
SIGMA/settings.py	leitura de FIREBASE_CREDENTIALS_FILE e SIGMA_APK_FILE

Rotas atuais:

Rota	Método	Função
/notificacoes/teste/	GET autenticado	abre a tela de teste
/notificacoes/apk/	GET autenticado	baixa o APK atual como SIGMA.apk
/notificacoes/dispositivos/registrar	POST autenticado com CSRF	associa o token ao usuário conectado
/notificacoes/teste/enviar/	POST autenticado com CSRF	envia mensagem fixa apenas ao próprio usuário

Tokens FCM são únicos. Um novo registro atualiza o proprietário e reativa o dispositivo. Tokens rejeitados pelo FCM como não registrados ou pertencentes a outro projeto são marcados como inativos. O sistema não expõe chave privada, token ou escolha livre de destinatário ao navegador.

3.6.3 Configuração do Firebase e do servidor

Para reconstruir a integração ou preparar outro ambiente:

1. No Firebase, usar o projeto `sigma-f611c` e manter cadastrado o aplicativo Android `br.com.indaiapapel.sigma`.
2. Baixar o `google-services.json` desse aplicativo e colocá-lo em `mobile/android/app/google-services.json`.
3. Gerar uma chave JSON para a conta de serviço utilizada pelo Firebase Admin.
4. Instalar a chave somente no servidor, fora de `/opt/SIGMA`, com proprietário `nexus` e permissão 600:

```
sudo install -o nexus -g nexus -m 600 serviceAccountKey.json /etc/sigma/firebase-admin.json
```

5. Manter em `/etc/sigma/sigma.env`:

```
FIREBASE_CREDENTIALS_FILE=/etc/sigma/firebase-admin.json
SIGMA_APK_FILE=/opt/SIGMA/artifacts/SIGMA.apk
```

6. Instalar as dependências bloqueadas, aplicar a migração e reiniciar o serviço:

```
cd /opt/SIGMA
/home/nexus/.local/bin/uv sync --frozen
.venv/bin/python manage.py migrate accounts --noinput
sudo systemctl restart sigma.service
systemctl is-active sigma.service
```

Ao substituir a conta de serviço, repetir a instalação do arquivo com permissão 600 e reiniciar `sigma.service`. Nunca copiar essa chave para `mobile/android/app`, para o APK, para o Git ou para uma tela do sistema.

Para entregar notificações, o servidor SIGMA precisa de acesso de saída à internet por HTTPS, porta 443, para os serviços Firebase/Google, e o dispositivo Android precisa de internet e dos serviços Google Play. Não é necessário publicar o SIGMA na internet nem liberar porta de entrada no firewall: as mensagens partem do servidor para o Firebase e são entregues ao celular pela conexão mantida pelo Google. O acesso às páginas do SIGMA continua restrito à rede corporativa ou VPN; sem internet a notificação fica pendente até o dispositivo recuperar conectividade.

3.6.4 Teste e novos disparos

O botão de sino no cabeçalho abre <https://sigma.indaialpapel.com.br/notificacoes/teste/>. A página oferece o download autenticado da versão atual do APK. Quando aberta pelo aplicativo Android, o botão **Ativar e enviar teste** solicita a permissão, registra o token e envia Teste do SIGMA somente para os aparelhos ativos do usuário conectado. O navegador comum pode baixar o APK e abrir a página, mas não possui o plugin nativo e não registra dispositivo.

Nenhum evento operacional dispara notificação automaticamente. Para adicionar um disparo, chamar o serviço no ponto da regra de negócio, passando um objeto de usuário obtido e validado pelo servidor:

```
from accounts.services.notificacoes import enviar_notificacao_usuario

enviar_notificacao_usuario(
    usuario=responsavel,
    titulo="Título da notificação",
    mensagem="Mensagem apresentada no Android",
    dados={"tipo": "identificador_do_evento"},
)
```

Não criar endpoint que aceite livremente token, destinatário, título ou mensagem enviados pelo navegador. Views, signals e workers devem determinar o destinatário no backend e chamar a função central. Um novo disparo Django não exige outro APK; recompilar apenas quando o comportamento nativo de recebimento ou abertura precisar mudar.

3.6.5 Geração e assinatura do APK

Para instalar dependências, sincronizar a plataforma Android e gerar o APK de produção assinado pelo terminal:

```
cd mobile
npm install
npm run apk:release
```

O script `mobile/build-release.ps1` lê `SIGMA_KEYSTORE_PASSWORD` do arquivo `.env` na raiz do projeto, configura o JDK e o Android SDK locais, executa `npx cap sync android`, compila com Gradle e copia o mesmo APK para:

```
mobile/dist/SIGMA.apk
artifacts/SIGMA.apk
```

Depois de gerar uma nova versão, enviar `artifacts/SIGMA.apk` para `/opt/SIGMA/artifacts/SIGMA.apk`. A view `baixar_apk_sigma` lê `SIGMA_APK_FILE`, exige usuário autenticado e responde o arquivo como anexo `SIGMA.apk`; o APK não passa pelo diretório `static` nem pelo `collectstatic`.

O APK de produção é assinado pela chave `mobile/keystore/sigma-release.jks`, alias `sigma`, RSA 4096, válida até 07/12/2053. A chave e sua senha não são versionadas e devem ser preservadas em armazenamento seguro externo ao repositório. Todas as atualizações precisam usar essa mesma chave. Em cada nova versão, incrementar o `versionCode`, ajustar o `versionName` quando necessário e executar novamente `npm run apk:release`.

O `.env` local e o arquivo de ambiente de produção contêm a variável em texto cru e não são versionados. Para preparar outra

máquina, copiar o mesmo keystore e configurar:

```
SIGMA_KEYSTORE_PASSWORD=senha-da-chave
```

A variável de ambiente do processo, quando definida, tem prioridade sobre o `.env`. A produção mantém a mesma variável no arquivo de ambiente para centralizar a configuração, embora a assinatura e a geração do APK sejam executadas na máquina de desenvolvimento.

4. Dados, bancos e modelo local

O SIGMA trabalha com mais de um banco porque cada um tem uma responsabilidade diferente. O PostgreSQL é o banco da aplicação: guarda o que o SIGMA controla. O Oracle ERP é a fonte oficial de OP, estoque, lote e movimentos. O Oracle Alchemy fornece informações de análise de bobina. Essa separação é fundamental para entender o sistema: nem tudo que aparece na tela está salvo localmente, e nem tudo que é salvo localmente já foi efetivado no ERP.

Em termos práticos:

- Se o usuário cadastrou um parâmetro, abriu chamado, gerou fila ou criou uma reunião, isso está no PostgreSQL.
- Se a tela mostra saldo de lote, situação de OP, operador ou movimento de estoque, normalmente isso veio do Oracle ERP.
- Se a tela mostra análise de bobina, flag de produção ou observação de qualidade vinda de máquina, isso vem do Alchemy.

4.1 PostgreSQL local

Configuração:

- Alias Django: `default`.
- Engine: `django.db.backends.postgresql`.
- Banco: `sigma`.
- Host nos ambientes de desenvolvimento e produção: PgBouncer local em `127.0.0.1:6432`; o PostgreSQL permanece em `127.0.0.1:5432`.
- Search path: `producao, manutencao, public`.
- PostgreSQL: 18.4, com TimescaleDB 2.28.3 habilitado.
- Limite PostgreSQL: `max_connections = 150`, com 3 conexões reservadas para superusuário e até 147 conexões normais. O PgBouncer limita o SIGMA a no máximo 100 conexões reais nesse banco, preservando margem para a trava direta dos workers e administração.

Uso:

O PostgreSQL é a memória operacional do SIGMA. Ele guarda os registros que a aplicação precisa controlar independentemente do ERP. Isso inclui cadastros, usuários, permissões, parâmetros, filas de integração, reuniões de qualidade, chamados de manutenção e histórico de sinais MQTT.

Nos ambientes de desenvolvimento e produção, o Django abre conexões curtas para o PgBouncer (`CONN_MAX_AGE = 0`). O PgBouncer, em pool por transação, devolve ao pool a conexão real do PostgreSQL ao final de cada transação. Ele mantém até 80 conexões normais e mais 20 de reserva, com teto de 100 conexões reais e até 300 clientes do SIGMA. Assim, aumentar threads ou usuários não multiplica diretamente as sessões no banco; requisições acima da capacidade aguardam uma conexão livre. O `search_path` é configurado no papel PostgreSQL do banco `sigma`, e não como parâmetro inicial da conexão. Os ciclos dos serviços em background também fecham suas conexões ao terminar.

No Windows de desenvolvimento, o serviço automático SIGMA-PgBouncer atende em `127.0.0.1:6432`; sua configuração fica em `C:\ProgramData\SIGMA\PgBouncer\pgbouncer.ini`. O arquivo local de autenticação é restrito ao serviço, administradores e usuário de desenvolvimento. O `.env` usa a porta 6432 para a aplicação e mantém `POSTGRES_DIRECT_PORT=5432` exclusivamente para a advisory lock global.

O PgBouncer fecha conexão real ociosa não reutilizada após 10 minutos (`server_idle_timeout = 600`). Também encerra cliente parado dentro de transação e query PostgreSQL que ultrapassem 5 minutos (`idle_transaction_timeout = 300` e `query_timeout = 300`), evitando que uma sessão travada consuma o pool indefinidamente.

A trava global dos workers é exceção: ela usa diretamente `127.0.0.1:5432`, fora do PgBouncer, pois o advisory lock precisa permanecer na mesma sessão durante a vida do processo. As conexões Oracle usadas por consultas específicas continuam sendo abertas e fechadas no próprio fluxo que as executa.

Cada conexão local do SIGMA informa sua origem em `application_name`: `sigma-web` para atendimento web, nomes específicos para MQTT e serviços em background, e `sigma-worker-lock` para a trava global. Isso permite separar, no PostgreSQL, conexões de tela, telemetria, integrações e coordenação de workers.

O processo também monitora a ocupação de conexões PostgreSQL e a fila do PgBouncer a cada 5 segundos. Ao alcançar 100 conexões de cliente, registra uma captura única no journal do `sigma.service` com total, limite do banco, PID, usuário, banco, origem/IP, `application_name`, estado, horários e a última query de cada sessão. O monitor só é rearmado quando o total cai para 70 ou menos, evitando repetição de logs durante o mesmo pico. Se algum cliente aguardar conexão no PgBouncer, registra uma captura única com clientes aguardando, tempo máximo de espera, clientes ativos, servidores ativos e conexões reais do pool; só rearma quando a fila zera. O journal é persistente no servidor; a consulta operacional é:

```
journalctl -u sigma.service --since "today 00:00" | grep -E 'POSTGRES_CONEXOES|PGBOUNCER_FILA'
```

Na faixa **Servidor** de `/services/status/`, PgBouncer: `usadas/100` é o consumo das conexões reais reservadas pelo pool para o banco `sigma`; `Esperando` mostra clientes que aguardam conexão livre. PostgreSQL: `total/147` é a contagem global das sessões reais do banco, incluindo a trava direta dos workers e conexões administrativas. Os dois indicadores medem camadas diferentes e devem ser lidos juntos. O botão **Detalhar conexões** abre o snapshot das sessões PostgreSQL com PID, rota, aplicação, estado, origem, evento de espera, horários e última query. A rota é comparada com `SHOW SERVERS` do PgBouncer: PgBouncer identifica uma conexão real mantida pelo pool e Direta identifica uma conexão que chegou ao PostgreSQL sem o pool, como a trava global dos workers.

No detalhe das conexões, Livre para reutilização significa a sessão ociosa que o PgBouncer pode emprestar para a próxima operação. Aguardando próximo comando é o estado técnico `ClientRead`: a conexão não está travada nem processando trabalho. Em uso indica query ou transação em execução; Parada dentro de transação merece atenção porque uma transação aberta não devolve a conexão ao pool.

A fila **Logs Apontamentos** processa no máximo 10 grupos de OP/estágio em paralelo (`LOGS_APONTAMENTOS_MAX_WORKERS`). Os demais grupos ficam na fila do executor e são enviados assim que um dos dez termina; esse limite impede que um lote grande crie uma thread por grupo.

As filas **Logs Apontamentos** e **Log Apontamento Componentes** usam a mesma chave de agrupamento — empresa, origem, OP, estágio e sequência — para reservar no máximo um registro da mesma OP/estágio por ciclo e evitar envio duplicado. A chave não determina a quantidade de threads: ambas podem enviar grupos diferentes em paralelo, limitadas a dez workers por fila (`LOGS_APONTAMENTOS_MAX_WORKERS` e `LOGS_APONTAMENTO_COMPONENTES_MAX_WORKERS`). Os grupos excedentes aguardam no executor; ao terminar uma chamada, inclusive com erro, a próxima chave inicia. Cada worker renova a atividade da fila antes de chamar o webservice, por até 180 segundos. O limite do ciclo da fila é 1.800 segundos (30 minutos); assim, a espera normal por vaga no limite de dez não é interpretada como travamento e o supervisor só solicita interrupção quando esse ciclo excede 30 minutos sem atividade válida. **Estorno Componentes** não agrupa por essa chave e processa um registro por vez em uma única execução.

A aplicação acessa esse banco principalmente pelo ORM do Django. Na prática, tabelas como apontamentos, liberações, pendências WMS, reuniões, chamados, cadastros e leituras de telemetria são manipuladas por modelos Python, não por SQL manual espalhado nas telas. Isso ajuda a manter validações, relacionamentos e regras de persistência centralizados no próprio projeto.

A principal razão de existir uma fila local é proteger a operação. Se o operador faz um apontamento e o ERP esta indisponível, o SIGMA não perde a ação; ele grava localmente e deixa pendente para envio posterior. O mesmo vale para WMS e algumas movimentações de qualidade.

4.2 Oracle ERP

Configuração:

- Alias Django: `oracle_erp`.
- Engine: `django.db.backends.oracle`.
- Host/DSN: `172.16.30.51/dbdev`.

Uso:

O Oracle ERP é tratado como fonte de verdade para dados industriais e administrativos do processo produtivo. Quando o SIGMA precisa saber se uma OP existe, qual a situação de um lote, quanto saldo existe em um depósito, qual operador esta cadastrado ou quais movimentos formam a rastreabilidade de um lote, ele consulta o ERP.

O SIGMA também usa o ERP como caminho para algumas consultas ao WMS via DBLINK. Isso ocorre, por exemplo, na consulta de endereço/local WMS da área vermelha e na importação de paletes para a tabela `USU_TPALWMS`.

Importante: o SIGMA consulta o Oracle diretamente, mas os movimentos que alteram o ERP são enviados preferencialmente por webservice Sapiens/Senior, respeitando as regras do ERP.

4.3 Oracle Alchemy

Configuração:

- Alias Django: `oracle_alchemy`.
- Engine: `django.db.backends.oracle`.
- Host/DSN: `172.16.30.10/dbprod`.

Uso:

O Alchemy aparece como base auxiliar de qualidade e máquina. Ele fornece dados de bobina e análise que ajudam a qualidade a decidir se um lote pode ser liberado, deve ir para área vermelha ou precisa de outra destinação.

O sistema também usa Alchemy para obter a última bobina da MP-III, enquanto MP-IV e MP-V são consultadas por endpoints web específicos dos recursos. O objetivo desses dados é manter a bobina atual dos recursos atualizada automaticamente.

4.4 Campos e tabelas customizadas no Oracle ERP

O SIGMA depende de campos `USU_` no Oracle ERP para complementar regras que não existem nos campos padrão do Senior. Esses campos funcionam como extensões da regra industrial: guardam situação customizada de lote, ligação entre movimentos, número de bobina, backup de configuração de baixa de componente e dados auxiliares usados pelas regras LSP dos webservices.

Essas customizações precisam ser tratadas como parte da arquitetura do sistema. Se um campo `USU_` estiver ausente, com tipo incorreto, sem carga ou com regra diferente da esperada, o erro pode aparecer como falha de apontamento, falha de rastreamento, lote não localizado, baixa duplicada de componente ou divergência entre ERP e WMS.

Campos customizados:

Campo	Tabela	Uso
<code>USU_ATRCCU</code>	<code>E011DEF</code>	Forma de atribuição do centro de custo no movimento de estoque: 0F para OP de fabricação, 0C para OP de consumo ou CF para centro de custo fixo.
<code>USU_CCUFIX</code>	<code>E011DEF</code>	Centro de custo fixo do motivo, usado quando <code>USU_ATRCCU</code> está configurado como CF.
<code>USU_UTIAVE</code>	<code>E011DEF</code>	Campo "Utiliza na Área Vermelha" da <code>F011DEF</code> ; quando está como S, permite que o motivo apareça na reunião de Área Vermelha.
<code>USU_SITLOT</code>	<code>E210DLS</code>	Situação personalizada do lote; valida pendência, exibe status, filtra busca e recebe ação dos webservices.
<code>USU_CODLIG</code>	<code>E210MVP</code>	Liga movimentos de estoque no rastreamento de lote; usar o código nativo geraria inconsistências fiscais nas transformações de produto.
<code>USU_NUMBOB</code>	<code>E900EQQ</code>	Número de bobina usado em apontamento, qualidade, rastreamento e localização de depósito do lote; liga ERP, Alchemy e recurso.
<code>USU_BXAORP</code>	<code>E900CMO</code>	Backup do <code>BXAORP</code> original do componente da OP para evitar baixa duplicada e restaurar o comportamento após a regra customizada.
<code>USU_QTDBOB</code>	<code>E900COP</code>	Quantidade de bobinas planejada/esperada da OP, exibida na tela de apontamento.

Campo	Tabela	Uso
USU_DESCRE	E725CRE	Descrição/local personalizado do centro de recurso usado no envio ao WMS.

A tabela customizada USU_TRESPED registra a reserva de pedidos para OPs. O Calendário de OPs a consulta somente para leitura, sempre pela empresa e filial do usuário logado, para mostrar o total comprometido e os pedidos reservados da OP. Ela não é uma tabela local do SIGMA nem é alterada pelo calendário.

Campos usados da USU_TRESPED:

Campo	Uso no Calendário de OPs
USU_CODEMP	Empresa da reserva; recebe a empresa da filial do usuário.
USU_CODFIL	Filial da reserva; recebe a filial do usuário.
USU_NUMORP	OP consultada.
USU_NUMPED	Pedido reservado exibido no detalhe da OP.
USU_QTDRES	Quantidade reservada; é somada por pedido e no total da OP.

A tabela customizada USU_TPALWMS é usada como ponte entre paleta WMS e componente ERP. Ela permite que o apontamento de componente receba um paleta lido na operação e descubra, no ERP, qual lote, produto, derivação e quantidade estão associados a esse paleta.

Campos principais da USU_TPALWMS:

Campo	Uso
USU_CODEMP	Empresa usada para localizar e atualizar paleta WMS. Faz parte da chave da importação.
USU_PALWMS	Código do paleta WMS recebido no apontamento de componente.
USU_QTDDIS	Quantidade disponível do paleta, usada como quantidade do componente e validação de saldo.
USU_CODLOT	Lote ERP vinculado ao paleta WMS.
USU_CODCMP	Código do componente vinculado ao paleta.
USU_DERCMP	Derivação do componente vinculado ao paleta.
USU_DATGER	Data de geração/processamento da importação ou leitura.
USU_HORGER	Hora de geração da importação do paleta, gravada em minutos do dia.

Campo	Uso
USU_ARMWMS	Armazém/origem WMS do palete importado.
USU_HORLOG	Hora de log atualizada quando o palete é lido ou apontado.
USU_LOGPRC	Log do processo do palete WMS, como leitura ou apontamento pelo APONTAR-COMPONENTE.

A tabela customizada USU_TBXACMP registra pendências de baixa de componente geradas pelos webservices de apontamento. Ela existe para separar o apontamento da OP da baixa posterior de componentes quando a regra precisa controlar a baixa de forma assíncrona, pelo fluxo automático do ERP e pelo webservice interno de baixa.

Campos principais da USU_TBXACMP:

Campo	Uso
USU_CODEMP	Empresa da pendência de baixa de componente.
USU_CODORI	Origem da OP vinculada a pendência.
USU_NUMORP	Número da OP vinculada a pendência.
USU_CODETG	Etapas da OP vinculada a pendência.
USU_CODCMP	Código do componente cuja baixa ficou pendente.
USU_DERCMP	Derivação do componente pendente.
USU_LOTDES	Lote destino/produto acabado associado a pendência.
USU_QTDUTI	Quantidade utilizada que deve ser baixada posteriormente.
USU_LOGINC	Log da inclusão da pendência, gravado no momento em que o apontamento cria o registro.
USU_CODCRE	Centro de recurso associado a pendência.
USU_CODLOT	Lote do componente a consumir. Quando não há lote informado, é gravado vazio; a regra de baixa trata espaço/vazio como ausência de lote.
USU_DATMOV	Data do movimento usado para registrar a pendência.
USU_HORMOV	Hora do movimento usado para registrar a pendência.
USU_SITPEN	Situação da pendência: 1 pendente, 2 em processamento, 3 sucesso e 4 erro.
USU_DATPRC	Data do processamento da baixa.

Campo	Uso
USU_HORPRC	Hora do processamento da baixa.
USU_USUPRC	Usuário/processo responsável pelo processamento.
USU_IDEUNI	Identificador único da pendência, gerado por GUID na inclusão e usado pela orquestração automática/webservice para processar o registro correto.

4.5 Índices personalizados

Os índices personalizados existem para sustentar consultas que seriam caras nas tabelas de rastreamento, paradas de máquina e OEE. Eles não mudam a regra de negócio, mas reduzem o custo de busca e ordenação em pontos onde o sistema precisa responder rápido.

No PostgreSQL local, os índices estão ligados principalmente ao histórico local do SIGMA: liberação de lote, paradas de máquina e planejado OEE.

ix_liberacao_lote_codlot

Uso: rastreamento público de lote. Busca eventos locais de qualidade pelo lote original e ordena por datager.

SQL:

```
CREATE INDEX ix_liberacao_lote_codlot
ON qualidade.liberacao_lote (codemp, codlot, datager);
```

ix_liberacao_lote_lottrf

Uso: rastreamento público de lote. Quando a URL recebe um lote transferido ou reclassificado, encontra o lote original pelo campo lottrf.

SQL:

```
CREATE INDEX ix_liberacao_lote_lottrf
ON qualidade.liberacao_lote (codemp, lottrf, datager);
```

idx_parada_recurso_fim

Uso: consultas de parada por recurso, principalmente para localizar parada em aberto ou histórico recente.

SQL:

```
CREATE INDEX idx_parada_recurso_fim
ON producao.paradas_maquina (recurso_id, fim);
```

idx_parada_tipo_fim

Uso: consultas de parada por tipo manual ou sinal, filtrando paradas abertas ou encerradas.

SQL:

```
CREATE INDEX idx_parada_tipo_fim
ON producao.paradas_maquina (tipo, fim);
```

idx_tlm_leitura_rec_data

Uso: histórico de telemetria por recurso, em ordem da leitura mais recente para a mais antiga.

SQL:

```
CREATE INDEX idx_tlm_leitura_rec_data
ON telemetria.leituras (recurso_id, coletado_em DESC);
```

A tabela `telemetria.leituras` é uma hypertable do TimescaleDB particionada por `coletado_em`. Sua identificação única é composta por `recurso_id` e `coletado_em`; nenhuma outra tabela possui referência direta a uma leitura. O intervalo padrão dos chunks é de sete dias. Não há política automática de retenção ou compressão configurada.

idx_apont_codemp_id

Uso: entrada nativa da tela Logs Apontamentos para usuários restritos à empresa, mantendo os registros mais recentes primeiro.

SQL:

```
CREATE INDEX idx_apont_codemp_id
ON producao.apontamento (codemp, id DESC);
```

idx_apont_fila_chave

Uso: ordenação e bloqueio da fila de apontamentos da mesma OP.

SQL:

```
CREATE INDEX idx_apont_fila_chave
ON producao.apontamento (status, codemp, origem, numorp, codetg, seqrot, id);
```

idx_comp_codemp_id

Uso: entrada nativa da tela Logs Componentes para usuários restritos à empresa, mantendo os registros mais recentes primeiro.

SQL:

```
CREATE INDEX idx_comp_codemp_id
ON producao.apontamento_componente (codemp, id DESC);
```

idx_comp_fila_chave

Uso: bloqueio da fila de componentes da mesma OP.

SQL:

```
CREATE INDEX idx_comp_fila_chave
ON producao.apontamento_componente (status, codemp, origem, numorp, codetg, seqrot);
```

idx_estorno_status_data

Uso: limpeza automática de estornos pendentes ou integrados com retenção superior a 60 dias.

SQL:

```
CREATE INDEX idx_estorno_status_data
ON producao.estorno_comp (status, adatfim);
```

idx_log_op_troca_id

Uso: paginação do Log Tempo Produção. Períodos ainda abertos aparecem antes dos fechados; dentro de cada grupo, os mais recentes aparecem primeiro.

SQL:

```
CREATE INDEX idx_log_op_troca_id
ON producao.logs_troca_op_ativa (horario_troca DESC, id DESC);
```

idx_lote_grupo_id

Uso: agrupamento e paginação da Consulta de Lotes por empresa, bobina, lote, produto e derivação.

SQL:

```
CREATE INDEX idx_lote_grupo_id
ON qualidade.liberacao_lote (codemp, numbob, codlot, codpro, codder, id DESC);
```

idx_lote_status_id

Uso: filtro de status e processamento da fila de liberação de lotes.

SQL:

```
CREATE INDEX idx_lote_status_id
ON qualidade.liberacao_lote (status, id DESC);
```

oeo_planeja_data_30140b_idx

Uso: consultas de OEE planejado por data.

SQL:

```
CREATE INDEX oeo_planeja_data_30140b_idx
ON public.oeo_planejado_diario (data);
```

No Oracle ERP, os índices estão ligados principalmente ao rastreamento público de lote. O rastreamento cruza eventos de apontamento, saldo atual e movimentos de estoque. Sem índices adequados, essas consultas podem ficar lentas porque as tabelas ERP possuem alto volume histórico.

I_NEX_E900EQQ_LOTE

Uso: rastreamento público de lote. Busca eventos de produção/apontamento ERP do lote e ordena por DATREA e HORREA.

SQL:

```
CREATE INDEX I_NEX_E900EQQ_LOTE
ON E900EQQ (CODEMP, CODLOT, DATREA, HORREA);
```

I_NEX_E210DLS_LOTE

Uso: rastreamento público de lote. Busca saldo atual ERP dos lotes encontrados no rastreamento.

SQL:

```
CREATE INDEX I_NEX_E210DLS_LOTE
ON E210DLS (CODEMP, CODLOT, CODDEP, CODPRO, CODDER);
```

I_NEX_E210MVP_LOTE_FULL

Uso: rastreamento público de lote. Busca movimentos ERP do lote original, obtém USU_CODLIG e ordena por DATDIG, HORDIG e SEQMOV.

SQL:

```
CREATE INDEX I_NEX_E210MVP_LOTE_FULL  
ON E210MVP (CODEMP, CODLOT, USU_CODLIG, DATDIG, HORDIG, SEQMOV);
```

I_NEX_E210MVP_LIG

Uso: rastreamento público de lote. Busca movimentos ERP ligados pelo mesmo USU_CODLIG e mantém transferências/reclassificações agrupadas no fluxo.

SQL:

```
CREATE INDEX I_NEX_E210MVP_LIG  
ON E210MVP (CODEMP, USU_CODLIG, DATDIG, HORDIG, SEQMOV);
```

I_NEX_E210MVP_LOTE_CODLIG

Uso: rastreamento público de lote. Atende o fallback por CODLIG quando o movimento não possui USU_CODLIG, buscando o lote original e ordenando por DATDIG, HORDIG e SEQMOV.

SQL:

```
CREATE INDEX I_NEX_E210MVP_LOTE_CODLIG  
ON E210MVP (CODEMP, CODLOT, ESTEOS, CODLIG, DATDIG, HORDIG, SEQMOV);
```

I_NEX_E210MVP_CODLIG

Uso: rastreamento público de lote. Busca movimentos ERP ligados pelo mesmo CODLIG quando o fallback é necessário, preservando a separação entre entrada e saída por ESTEOS.

SQL:

```
CREATE INDEX I_NEX_E210MVP_CODLIG  
ON E210MVP (CODEMP, CODLIG, ESTEOS, DATDIG, HORDIG, SEQMOV, CODLOT);
```

I_NEX_E900EQQ_OP_RECORSO

Uso: rastreamento público de lote. Busca a máquina/recurso da OP a partir dos movimentos de estoque E210MVP.ORIORP e E210MVP.NUMDOC, usando a menor sequência de operação com CODCRE preenchido.

SQL:

```
CREATE INDEX I_NEX_E900EQQ_OP_RECORSO  
ON E900EQQ (CODEMP, CODORI, NUMORP, CODCRE, SEQEQQ);
```

4.6 Principais modelos locais do projeto

Esta seção descreve as principais entidades que o SIGMA controla localmente. Não é necessário memorizar os nomes técnicos; o importante é entender que cada entidade representa uma parte da operação que o sistema precisa guardar para funcionar.

Os modelos locais podem ser divididos em cinco grupos:

1. Estrutura e parâmetros: definem a organização da fábrica e como cada filial/centro/recurso deve se comportar.
2. Produção: guardam sequenciamento, apontamentos e filas de envio ao ERP.
3. Qualidade: guardam reuniões, liberações de lote e filas WMS.
4. Manutenção: guardam chamados, ordens de serviço e interações.
5. Telemetria: guardam configuração HTTP, sensores, vínculos por recurso e leituras interpretadas.

4.6.1 Accounts

O grupo **Accounts** representa a base administrativa do SIGMA. Ele define quem usa o sistema, em qual filial trabalha, quais recursos existem, quais centros produtivos organizam esses recursos e quais parâmetros devem ser aplicados nos fluxos operacionais. Sem essa camada, os módulos de produção, qualidade, manutenção e OEE não conseguem decidir corretamente onde buscar saldo, para onde movimentar lote, qual recurso recebe apontamento ou qual calendário deve ser usado.

Um ponto importante é que esses modelos não são apenas “cadastro”. Muitos deles interferem diretamente na regra de negócio. Por exemplo, **ParametrosFilial**, **ParametrosCentroRecurso** e **ParametrosRecurso** determinam depósitos, limites e comportamentos que aparecem nos fluxos de apontamento, liberação de lote, área vermelha e OEE. O cadastro de **Recurso** também é usado por MQTT, manutenção, sequenciamento, OP ativa e cálculo de planejado.

No cadastro de recursos, a aba OEE possui campos que foram mantidos para evolução futura do módulo de OEE, como metas e quantidade de pessoas. No funcionamento atual, os campos dessa aba que interferem diretamente no fluxo produtivo são **view_id**, **aponta_parada**, **exibir_jus** e **permite_parada_manual**. O **view_id** define qual versão da tela de apontamento será aberta para o recurso quando o operador entrar por uma OP/código de barras e define se o sequenciamento base pode controlar a OP ativa: as Views 0, 1 e 2 usam o fluxo de uma OP; a View 3 é reservada ao fluxo Multi-OP e controla suas alocações na própria tela. Os campos **aponta_parada** e **exibir_jus** atuam em conjunto no modal de justificativa e no bloqueio visual de apontamento. **permite_parada_manual** autoriza a abertura manual somente se **aponta_parada** também estiver ativo; a permissão **Pode Alterar Paradas** também autoriza essa abertura, inclusive quando recebida por grupo, mas nunca ignora **aponta_parada**.

A aba **Motivos de Parada** do recurso mantém a abrangência dos motivos ERP permitidos para aquele recurso. O sistema deriva **codemp** da empresa da filial do recurso, lista os grupos ativos de **USU_T018GPM** (**USU_CODGPM**, **USU_DESGPM**, **USU_SITGPM**='A') e, após a escolha, lista os motivos ativos de **E018MTV** (**CODMTV**, **DESMTV**, **SITMTV**='A') ligados ao grupo em **USU_T018MVP** (**USU_CODEMP**, **USU_CODGMP**, **USU_CODMTV**). As consultas usam a conexão Oracle padrão, que já seleciona o schema do ambiente. O vínculo local guarda **id_recurso**, **codemp**, **codgpm** e **codmtv**; a chave única impede repetir a combinação recurso, empresa, grupo e motivo ERP. A mesma aba também copia os vínculos de um recurso ativo escolhido por empresa: os códigos são gravados para a empresa do recurso de destino somente quando continuam ativos no ERP, sem duplicar vínculos já existentes. O botão **Sincronizar motivos de todos os recursos** remove os vínculos locais cujo grupo ou motivo não esteja ativo e ligado em **USU_T018MVP**; se a consulta ERP falhar para alguma empresa, não remove vínculo algum. Nas telas de apontamento, esses vínculos são a lista visual de motivos disponível para justificar uma parada aberta.

Modelo	Tabela	Finalidade
CustomUser	accounts_customuser	Usuário do sistema, filial, operador ERP e página inicial.
Empresa	empresas	Empresa, código ERP, lote atual e status ativo.
Filial	filial	Filial vinculada a empresa.
ParametrosFilial	parametros_filial	Parâmetros integração/qualidade por filial.
Departamento	departamentos	Agrupamento organizacional.
Setor	setores	Setor vinculado a departamento.
CentroRecurso	centros_recursos	Centro produtivo, código e código integrador ERP.
ParametrosCentroRecurso	parametros_centros_recursos	Depósitos e parâmetros por centro.
Recurso	recursos	Recurso produtivo, configurações OEE/apontamento e bobina atual.
ParametrosRecurso	parametros_recursos	Parâmetros específicos do recurso.

Modelo	Tabela	Finalidade
Tara	taras	Cadastro de taras.
RecursoTara	recurso_taras	Relacao recurso x tara.
MotivoAbrangencia	motivos_abrangencia	Grupo e motivo ERP ativos permitidos por recurso e empresa, identificados por CODGPM e CODMTV.
LogTrocaOPAtiva	logs_troca_op_ativa	Histórico de OP ativa por recurso, guardando a OP em campos decompostos (origem, op, estagio, seqrot), horário da troca/saída, operador e status/log/data_hora de integração.
TurnoBase	turnos_base	Turno padrão.
TurnoRecurso	turnos_recursos	Turno aplicado ao recurso e dias da semana.
Calendario	calendario	Calendário por filial.
CalendarioEvento	calendario_eventos	Feriado, parada e evento.
HoraExtraPlanejada	horas_extras_planejadas	Horas extras planejadas por recurso.
ParadaMaquina	producao.paradas_maquina	Parada física do recurso, com início, fim, código do operador, usuário, tipo e data/hora. Relaciona-se a um ou mais períodos de LogTrocaOPAtiva afetados.
JustificativaParada	producao.justificativas_paradas	Justificativas sequenciais da parada, com motivo ERP, início parcial e tempo. A última justificativa de uma parada aberta permanece sem tempo até ser substituída ou até o fim da parada.
PacoteTempoERP	producao.pacotes_tempo_erp	Corte local de um período de LogTrocaOPAtiva para envio de tempos ao ERP, com início/fim reais, status, log e data/hora do retorno.
ItemPacoteTempoERP	producao.itens_pacote_tempo_erp	Produção ou parada pertencente ao pacote, com operador, motivo de parada e data/hora inicial e final que serão enviadas ao ERP.
OEEPlanejadoDiario	public.oe_planejado_diario	Minutos planejados por recurso/dia.

4.6.2 Telemetria

O módulo **telemetria** é independente dos demais cadastros, mas usa `accounts.Recurso` como referência. Seus dados ficam no schema PostgreSQL **telemetria**; o recurso continua na tabela `public.recursos`. A regra automática de parada pertence ao domínio de produção, fica no schema `producao` e também se relaciona diretamente ao recurso. Não há MQTT, OPC-UA, Modbus ou autenticação HTTP neste módulo.

A configuração HTTP é individual por recurso. Ela define URL, coleta ativa, timeout, pausa após sucesso e espera após erro.

O cadastro mestre de sensores usa código fixo global e único; nome amigável pode mudar ou se repetir. Um sensor pode ser vinculado a vários recursos, mas somente uma vez em cada recurso, e cada posição da resposta HTTP só pode ser usada uma vez no mesmo recurso.

Modelo	Tabela	Finalidade
ConfiguracaoColetaHTTP	telemetria.configuracoes_http_recurso	Uma configuração HTTP por recurso, incluindo URL, timeout, pausas e último erro.
Sensor	telemetria.sensores	Cadastro mestre de código, nome, tipo (decimal, inteiro, booleano ou texto), unidade e situação.
SensorRecurso	telemetria.sensores_recursos	Vínculo do sensor ao recurso, posição HTML e regra de monitoramento/tolerância daquele recurso.
LeituraTelemetria	telemetria.leituras	Leitura interpretada em JSONB, identificada por recurso e data/hora da coleta.
RegraParadaRecurso	producao.regras_parada_recursos	Regra automática de parada, única por recurso, com situação e árvore JSONB de grupos e condições.

Na aba **Telemetria** do cadastro do recurso, o usuário configura a comunicação e os vínculos, além de consultar o cache do coletor com o último snapshot interpretado e a última leitura efetivamente salva. A aba **Parada Automática** concentra a regra de parada por telemetria: ela pode ser ativada ou desativada e é montada visualmente com condições, grupos E, OU e NÃO, grupos aninhados e reordenação dos itens; a tela só oferece sensores ativos vinculados ao próprio recurso. A própria tela explica que uma condição testa um sensor e que um grupo combina resultados, e exibe em modo somente leitura o JSON montado que será salvo. O cadastro mestre **Sensores** fica no submenu de Recursos e permite listar, incluir, editar, ativar/desativar e excluir sensores que não estejam vinculados.

Campos do cadastro de usuário (CustomUser):

Campo	Tela	Onde aparece	Uso no sistema
filial	Filial	Cadastro/edição de usuário e lista de usuários.	Define a empresa e filial do usuário logado. A filial limita empresas e centros exibidos nas telas, filtra logs e registros para usuários sem perfil staff, busca parâmetros de liberação de lote e Área Vermelha, e valida acesso a etiquetas de outra empresa.
idintegracao	Idintegracao ou ID Integração	Cadastro/edição e lista de usuários.	Identificador enviado ao ERP nas ações de qualidade. É usado em Área Vermelha, liberação, refugo e reclassificação de lote. No SOAP alimenta usuRes. Sem esse valor, a integração retorna erro de usuário sem ID configurado.

Campo	Tela	Onde aparece	Uso no sistema
idoperador	ID Operador	Cadastro/edição de usuário.	Campo cadastral sem uso operacional atual. O operador do apontamento é digitado na tela, validado no ERP em E9060PE e gravado em LogTrocaOPAtiva.id_operador.

4.6.3 Produção

Os modelos de produção guardam a trilha local do que a fábrica tentou executar. Eles são essenciais porque o ERP não é chamado de forma cega e descartável; antes ou durante a integração, o SIGMA cria registros locais que permitem consultar, acompanhar e reprocessar.

O Sequenciamento organiza a ordem planejada das OPs por recurso. O Apontamento registra apontamentos de OP e controla se foram enviados com sucesso ao ERP. O ApontamentoComponente faz o mesmo para consumo/baixa de componentes, podendo também gerar ajuste no WMS quando o componente vem de palete WMS. O EstornoComponente controla o caminho inverso, quando uma baixa de componente precisa ser estornada no ERP devido a um ajuste no apontamento.

Modelo	Tabela	Finalidade
Sequenciamento	producao.sequenciamento	Sequência local de OPs por recurso.
Apontamento	producao.apontamento	Fila/log local de apontamentos de OP.
ApontamentoComponente	producao.apontamento_componente	Fila/log local de apontamentos de componentes.
EstornoComponente	producao.estorno_comp	Fila/log local de estornos de componentes.

4.6.4 Qualidade

Os modelos de qualidade guardam decisões e pendências que impactam lote, estoque e WMS. A qualidade não apenas consulta informações: ela decide se um lote pode seguir, se deve ficar em área vermelha, se será refugo, se será reclassificado ou se precisa gerar movimento externo.

A Reuniao representa a análise da área vermelha e seus participantes. A LiberacaoLote registra decisões de liberação, refugo ou reclassificação que depois podem virar movimentação no ERP. A fila WMS_IntegraçãoOP e o controle local do que precisa ser enviado ao WMS, seja novo lote, ajuste de estoque ou outro tipo suportado pelo fluxo.

Modelo	Tabela	Finalidade
Reuniao	qualidade.reuniao	Reunião de área vermelha.
ReuniaoParticipante	qualidade.reuniao_participantes	Participantes da reunião.
ObservacaoEtiqueta	qualidade.observacao_etiqueta	Observações selecionáveis na etiqueta.
LiberacaoLote	qualidade.liberacao_lote	Liberação, refugo ou reclassificação de lote.

Modelo	Tabela	Finalidade
WMS_IntegraçãoOP	qualidade.wms_integracao_op	Fila de envio ao WMS.

4.6.5 Manutenção

Os modelos de manutenção registram solicitacoes, ordens de serviço e histórico de comunicação. A relacao com Recurso e importante porque o chamado nasce ligado ao equipamento ou ponto produtivo afetado. Isso permite consultar histórico por recurso e direcionar atendimento.

As interações funcionam como linha do tempo. Elas indicam o que foi informado, quem respondeu, quais anexos ou descrições foram acrescentados e como a OS ou chamado evoluiu até encerramento.

Modelo	Tabela	Finalidade
Chamado	chamado	Chamado de manutenção por recurso.
Interacao_Chamado	chamado_interacao	Histórico de interações do chamado.
OrdemServico	ordem_servico	Ordem de serviço.
Interacao_OS	ordem_servico_interacao	Histórico de execução/interação da OS.

5. Navegacao e rotas principais

As rotas indicam os endereços internos das funcionalidades no sistema web. Elas servem para mostrar como o sistema está organizado para o usuário, não para explicar código.

As telas principais ficam agrupadas por área:

- Cadastros e administração na raiz do sistema.
- Produção em /producao/.
- Qualidade em /setores/qualidade/.
- Manutenção em /setores/manutencao/.
- Telemetria em /telemetria/, vinculada ao cadastro de recursos.
- WebSocket em /ws/, usado por recursos em tempo real.

5.1 Raiz

As rotas de raiz são os pontos de entrada dos grandes blocos do sistema. Elas não representam uma tela única, mas a divisão principal de navegacao. Quando o usuário acessa produção, qualidade ou manutenção, o Django encaminha para o conjunto de URLs daquele módulo.

Rota	Módulo
/	accounts.urls
/producao/	producao.urls
/setores/	setores.urls
/select2/	django_select2
/telemetria/	telemetria.urls

Rota	Módulo
------	--------

5.2 Telemetria

Rota	Finalidade
/telemetria/sensores/	Cadastro mestre de sensores.
/telemetria/recursos/<id>/configurar/	Grava a configuração HTTP e os vínculos da aba Telemetria do recurso.

5.3 Produção

As rotas de produção concentram as telas usadas no acompanhamento e registro da operação produtiva. Elas permitem apontar OP, consultar status de recursos, verificar filas de envio ao ERP, tratar componentes, estornar componentes e organizar sequenciamento. Para suporte, essas rotas ajudam a localizar rapidamente onde um erro operacional se manifesta.

Rota	Finalidade
/producao/apontamentos/	Tela base de apontamento.
/producao/status-recursos/	Painel de status de recursos.
/producao/logs-apontamentos/	Logs/fila de apontamento OP.
/producao/log-tempo-producao/	Histórico local de períodos produtivos, paradas e justificativas.
/producao/log-tempos-erp/	Logs/fila dos pacotes de tempos enviados ao ERP.
/producao/logs-apontamento-componentes/	Logs/fila de apontamento de componentes (v1/v2, via ApontamentoComponente).
/producao/logs-baixa-componentes/	Logs/fila de baixa de componentes gerada pela tela Multi-OP (View 3), via TratarBaixa.
/producao/estorno-componentes/	Estornos de componentes.
/producao/sequenciamento/	Sequenciamento de OPs.

5.4 Qualidade

As rotas de qualidade concentram as decisões que afetam lote e estoque. A mesma área permite liberar lote, abrir ou tratar área vermelha, consultar lote destinado, rastrear origem/destino e acompanhar pendências de WMS. Isso mostra que qualidade não é apenas uma consulta: ela participa da movimentação controlada do material.

Rota	Finalidade
/setores/qualidade/liberar-lotes/	Liberação de lotes.
/setores/qualidade/area-vermelha/	Reunião/área vermelha.
/setores/qualidade/consulta-lote/	Consulta de lotes destinados.
/setores/qualidade/consulta-lote/rastreamento/	Rastreamento de lote por empresa e lote; disponível também em Produção > Relatórios.
/setores/qualidade/integracao-wms/	Fila WMS.

5.5 PCP

O PCP disponibiliza uma visão de calendário das OPs por estágio. A tela é somente de consulta: usa as datas do ERP para identificar em quais dias cada estágio estará em produção e permite restringir a visualização por máquina, origem, estágio, produto-derivação e situação da OP.

Rota	Finalidade
/setores/pcp/calendario-ops/	Tela do Calendário de OPs.
/setores/pcp/calendario-ops/eventos/	Consulta JSON dos eventos exibidos no período e filtros selecionados.
/setores/pcp/calendario-ops/detalhes/	Consulta JSON do detalhamento do cálculo de comprometimento da OP selecionada.

5.6 Manutenção

As rotas de manutenção conectam solicitacao, atendimento e recurso. O QR Code por recurso permite que o usuário abra chamado a partir do ponto físico de operação, reduzindo erro de seleção de equipamento. As ordens de serviço complementam o chamado com planejamento, responsáveis e execução.

Rota	Finalidade
/setores/manutencao/	Lista chamados.
/setores/manutencao/chamados/novo/	Abre chamado.
/setores/manutencao/recurso/<id>/qrcode/	Gera QR Code para recurso.
/setores/manutencao/os/	Lista ordens de serviço.
/setores/manutencao/os/abrir/	Abre ordem de serviço.

5.7 WebSocket

As rotas WebSocket existem para dados que precisam chegar na tela sem recarregar a página. No caso da balança, a leitura recebida por MQTT no servidor pode ser enviada para a tela aberta do recurso. No caso de OP, a tela pode receber atualizações relacionadas a OP ativa.

Rota	Consumidor	Finalidade
/ws/balanca/<recurso_id>/	ConsumidorBalanca	Atualização da balança em tempo real.
/ws/op/<codbar>/	OPConsumer	Atualização de OP ativa.

6. Integrações externas e contratos de comunicação

As integrações são a parte mais sensível do SIGMA. Elas conectam a operação local com sistemas que possuem regras e disponibilidade próprias. Por isso, o SIGMA evita depender de uma única chamada imediata para concluir uma operação crítica. Sempre que possível, ele registra localmente, tenta enviar, interpreta o retorno e deixa log para reprocessamento.

Existem dois tipos de integração:

1. Consulta: o SIGMA busca informações em outro sistema para exibir, validar ou decidir.
2. Envio: o SIGMA envia uma solicitação para outro sistema efetivar uma ação.

Exemplo: consultar saldo de lote no Oracle ERP e uma consulta. Enviar `MovimentarEstoque` para o Sapiens e um envio.

6.1 ERP Oracle - consulta direta

O Oracle ERP é consultado diretamente quando o SIGMA precisa ler dados que já existem no ERP. Essas consultas alimentam telas e validações, mas em geral não fazem a movimentação principal do processo. A movimentação é feita por webservice.

Essa abordagem permite que o SIGMA mostre dados atualizados do ERP sem duplicar toda a base localmente. Ao mesmo tempo, mantém as alterações importantes passando pelas regras do ERP/Sapiens.

Principais tabelas usadas:

Tabela	Uso
E900COP	OPs, situação da OP, sequenciamento e encerramento.
E900QDO	Dados da OP/quantidade.
E9000OP	Operações/roteiro/centro de recurso.
E900CMO	Componentes da OP.
E900EQQ	Eventos/apontamentos, bobina, lote, recurso, data/hora.
E210DLS	Saldo de lote por depósito/produto/derivacao.
E210MVP	Movimentos de estoque e rastreabilidade.
E725CRE	Centro de recurso ERP.
E075PRO / E075DER	Produto e derivacao.

Tabela	Uso
E9060PE	Operadores.
E011DEF	Motivos/defeitos de área vermelha.
USU_TPALWMS	Paletes WMS importados no ERP.
USU_TRESPED	Reservas de pedidos por OP, usadas para mostrar quantidade comprometida no Calendário de OPs.

6.2 ERP Sapiens/Senior - SOAP

O Sapiens/Senior SOAP é usado quando o SIGMA precisa pedir ao ERP para executar uma ação de negócio que não pode ser resolvida apenas no banco local. Cada webservice existe para um motivo operacional específico: registrar produção, controlar consumo de componente, movimentar lote, tratar refugo/reclassificação, enviar lote para área vermelha ou desfazer baixa de componente.

O SIGMA monta um JSON com os dados da operação e envia esse JSON dentro de um envelope SOAP. O ERP processa a regra customizada em LSP e devolve um retorno. O SIGMA interpreta esse retorno para decidir se o registro local deve virar integrado ou voltar para pendente.

O ponto mais importante é que esses webservices não são apenas “envio de dados”. Eles executam regras do lado do Senior: chamam rotinas padrão do ERP, ajustam situação customizada de lote, controlam baixa automática de componentes, gravam pendências internas para baixa posterior e preservam ligações de rastreabilidade.

Base:

```
SAPIENS_URL_BASE/g5-senior-services/sapiens_Synccustom.senior.man.producao
```

Formato:

- HTTP POST.
- Content-Type application/soap+xml; charset=ISO-8859-1.
- Envelope SOAP.
- Usuário e senha vindos de ambiente.
- Payload JSON colocado em CDATA no campo `wdados`.

As respostas desses webservices podem voltar em codificação diferente do UTF-8 usado pela aplicação. Por isso, o SIGMA centraliza a leitura de resposta HTTP em utilitário próprio de codificação. Esse tratamento evita que uma resposta SOAP válida seja interpretada com caracteres quebrados ou gere erro ao gravar log da fila.

6.2.1 Webservices enviados e motivo operacional

Serviço	Quando o SIGMA usa	Motivo do envio	O que a regra Senior faz
Apontamentos	Apontamento normal de OP.	Efetivar no ERP a produção apontada na fábrica, com quantidade boa, refugo, lote, bobina, recurso, operador e horário.	Executa o apontamento da OP, evita duplicidade do mesmo apontamento, controla início/fim quando necessário, desliga temporariamente a baixa automática da OP, restaura a configuração original e cria pendências de baixa de componentes em USU_TBXACMP com USU_SITPEN=1 e USU_IDEUNI gerado por GUID. A quantidade da pendência segue o modelo técnico: fixa quando configurada ou proporcional à quantidade apontada. Também marca o lote apontado como pendente na situação customizada do ERP.
AumentarApontamento	Correcao/aumento de apontamento já existente.	Acrescentar quantidade a uma OP/lote sem repetir o fluxo operacional completo da tela de apontamento.	Reaproveita a lógica de apontamento para incrementar quantidade, evita duplicidade, calcula componentes proporcionais, preserva a configuração original de baixa automática e gera pendências de baixa em USU_TBXACMP com status pendente e identificador único.
ApontamentoComponente	Apontamento de componente informado pela operação, normalmente a partir de lote ERP ou palete WMS resolvido pelo SIGMA.	Registrar no ERP que determinado componente foi consumido para a OP e, ao mesmo tempo, apontar a OP pela quantidade líquida correspondente quando aplicável.	Valida saldo do lote do componente, calcula quantidade líquida considerando perda de processo, aponta a OP, grava pendências proporcionais de componentes e grava também o componente recebido para baixa posterior, usando USU_SITPEN=1, USU_LOGINC e USU_IDEUNI.
ApontamentoTempos	Pacote consolidado de produção e paradas de uma OP/recurso.	Registrar no ERP os períodos de produção e as paradas com operador, motivo e horários.	Recebe a chave da OP (empresa, origem, op, estagio, roteiro, máquina) e as listas producoes e paradas. Exige OP liberada ou em andamento, recurso com centro de custo, nenhum início de OP sem fim no ERP e exatamente uma Produção. O início da Produção identifica duplicidade. Produção e Paradas exigem operador com superior cadastrado; se a Parada tiver outro superior, usa o superior da Produção. O pacote é uma única transação: erro em um item impede a efetivação dos demais.

Serviço	Quando o SIGMA usa	Motivo do envio	O que a regra Senior faz
Regra orquestradora 230	Processamento automático Senior para pendências USU_TBxACMP.	Separar o disparo da baixa do momento em que o apontamento cria a pendência.	Segue o fluxo automático da IPEL: busca pendências com USU_SITPEN=1, respeita o paralelismo configurado em E000PDV, marca como USU_SITPEN=2 e envia os registros de forma assíncrona para o webservice interno BaixaComponenteERP, informando o USU_IDEUNI.
BaixarComponentes	Regra do webservice interno BaixaComponenteERP, chamada pela orquestração automática.	Efetivar a baixa de componentes que ficaram pendentes após apontamento de OP ou apontamento de componente.	Recebe ideUni, busca a pendência por USU_IDEUNI, valida família, centro de custo, depósito, saldo e lote, libera o componente na OP, executa a baixa e atualiza a pendência para USU_SITPEN=3 em sucesso ou USU_SITPEN=4 em erro. Também grava log de processamento com CodPrc=3 e IdePrc=BXC.
MovimentarEstoque	Liberação normal de lote ou envio de lote para Área Vermelha quando o produto/lote permanece o mesmo e muda o depósito/local.	Movimentar estoque no ERP entre depósitos e atualizar a situação customizada do lote.	Executa movimentação de estoque quando origem é destino exigem movimento. Se não houver movimento necessário, apenas trata a situação. Ao final, grava em E210DLS.USU_SITLOT a situação enviada pelo SIGMA.
TransferenciaProduto	Refugo ou reclassificação de lote.	Transformar o saldo de um produto/derivacao/lote em outro produto/derivacao/lote, preservando rastreabilidade.	Executa transferência entre produtos, calcula valor de entrada com base no preço médio da derivacao destino, aplica centro de custo conforme regra fiscal, preserva a ligação de rastreabilidade em campo customizado e atualiza a situação customizada do lote original e do lote destino.
EstornaComp	Estorno de componente.	Desfazer consumo de componente que já havia sido baixado contra uma OP.	Localiza movimento anterior do componente, prepara a OP para permitir o estorno com lote/depósito corretos e chama a rotina de estorno de componentes do ERP.

Serviço	Quando o SIGMA usa	Motivo do envio	O que a regra Senior faz
TratarBaixa	Baixa de componente gerada pela tela Multi-OP (View 3), um registro por bobina de consumo debitada em cada apontamento.	Registrar no ERP o consumo de bobina de matéria-prima da máquina, incluindo o caso de repesagem (bobina retirada da máquina após já ter sido parcialmente consumida).	Não executa a baixa física: grava a pendência em <code>USU_TBXCMP</code> com <code>USU_SITPEN=1</code> , mesma tabela usada por <code>Apontamentos/ApontamentoComponente</code> . A quantidade enviada só é usada quando <code>repesagem=N</code> e <code>consumototal=N</code> ; quando <code>repesagem=S</code> , a regra ignora a quantidade recebida e recalcula ela mesma como saldo atual do lote menos 1 (preserva saldo 1 para não duplicar repesagem); quando <code>consumototal=S</code> , usa todo o saldo atual do lote. Rejeita quando os dois indicadores vêm S ao mesmo tempo. A execução física continua na regra orquestradora 230, exatamente como as demais pendências de <code>USU_TBXCMP</code> .

6.2.2 Por que o envio de apontamento não baixa componente diretamente ?

No apontamento de OP, o SIGMA envia ao ERP a produção realizada. A regra Senior precisa lidar com um problema operacional: Foi realizado fisicamente a produção da OP que logo imediatamente pode ser usado para movimentações ou outras operações, porém pode ocorrer de não ter estoque dos componentes e por conta disso o apontamento retornaria erro e não seria efetivado no sistema.

Por isso, a regra de apontamento desliga temporariamente a baixa automática dos componentes da OP, executa o apontamento da produção e depois restaura a configuração original. Em seguida, grava pendências de baixa em `USU_TBXCMP`. Essas pendências representam os componentes que precisam ser baixados em etapa controlada, com validação de saldo, lote, depósito e regra de componente.

Esse desenho separa duas responsabilidades:

1. O apontamento da OP registra a produção do produto acabado.
2. A baixa de componentes consome os materiais associados, com controle posterior e rastreabilidade.

No `ApontamentoComponente`, a regra vai além: ela também recebe explicitamente o componente informado pela operação. Esse componente pode vir de leitura de lote ou de paleta WMS resolvido previamente pelo SIGMA. A regra valida se existe saldo suficiente e grava a baixa desse componente como pendência, mesmo que ele não esteja marcado como componente de baixa automática original da OP.

6.2.3 Por que existe a orquestração automática da baixa de componentes ?

A baixa de componentes foi separada em duas partes dentro do Senior. Os webservices de apontamento (`Apontamentos`, `AumentarApontamento`, `ApontamentoComponente` e `TratarBaixa`) não executam a baixa no mesmo momento em que apontam a OP. Eles apenas criam pendências em `USU_TBXCMP`, com `USU_SITPEN=1`, log de inclusão em `USU_LOGINC` e um identificador único `USU_IDEUNI` gerado por GUID.

`TratarBaixa` existe como ponto de entrada próprio, em vez de reaproveitar `ApontamentoComponente`, porque a origem do consumo é diferente na tela Multi-OP: não é o operador informando um componente lido por etiqueta ou paleta, é o rateio de bobinas de matéria-prima da máquina calculado pelo SIGMA a cada apontamento de produção. O webservice recebe direto o resultado desse rateio — quanto foi debitado de cada bobina — e apenas grava a pendência; a mesma regra também sabe tratar o caso de **repesagem** (quando uma bobina que já teve consumo real é retirada da máquina antes de zerar),

recalculando a quantidade a baixar a partir do saldo real do lote no ERP em vez de confiar no valor local, que pode estar defasado entre o momento da retirada física e o envio da baixa.

A regra orquestradora 230 é a responsável por consumir essa fila dentro do ERP. Ela segue o fluxo padrão da IPEL para processamentos automáticos: busca pendências com `USU_SITPEN=1`, respeita o nível de paralelismo configurado em `E000PDV` pela chave `CUSTOM.SENIOR.MAN.PRODUCAO.BAIXACOMPONENTEERP`, marca os registros como `USU_SITPEN=2` e envia os registros de forma assíncrona para o webservice interno `BaixaComponenteERP`, informando o `USU_IDEUNI` da pendência.

O `BaixaComponentes` é a regra do webservice interno. Ele recebe `BaixaComponenteERP.pendencias.ideUni`, busca a pendência correspondente em `USU_TBXACMP` por `USU_IDEUNI` e mantém a lógica de baixa: valida família, centro de custo, depósito, saldo e lote; libera o componente na OP e executa a baixa no ERP. Quando a pendência informa lote, esse lote é obrigatório: se não houver saldo nele, a baixa falha sem substituir por outro. Sem lote informado, a regra procura um lote disponível.

O resultado da baixa volta para a própria pendência:

- `USU_SITPEN=1`: pendente.
- `USU_SITPEN=2`: em processamento.
- `USU_SITPEN=3`: processado com sucesso.
- `USU_SITPEN=4`: erro.

Os dados do processamento ficam em `USU_DATPRC`, `USU_HORPRC` e `USU_USUPRC`. O log técnico do processamento é gravado pelo serviço interno `interno.custom.senior.logs.GravarLog`, com `CodPrc=3` e `IdePrc=BXC`. O campo adicional do log inclui origem, OP, componente e quantidade, permitindo rastrear qual pendência foi processada e qual foi o retorno.

Quando uma pendência precisa voltar para a fila, o fluxo de reabilitação de pendências da IPEL também trata esse processo. Para processos com `CodPrc=3` e `IdePrc=BXC`, a reabilitação volta a pendência de componente para `USU_SITPEN=1` e limpa `USU_DATPRC`, `USU_HORPRC` e `USU_USUPRC`, permitindo novo processamento pelo fluxo automático.

6.2.4 Webservices de lote e situação customizada

Os webservices `MovimentarEstoque` e `TransferenciaProduto` tratam destinação de lote. Eles não existem apenas para mudar saldo; também atualizam a situação customizada do lote no ERP.

`MovimentarEstoque` é usado quando a operação muda depósito/local do mesmo produto e lote. E o caso de liberação normal e envio para Área Vermelha. Se origem é destino já forem equivalentes, a regra não precisa executar movimento, mas ainda pode atualizar a situação do lote.

`TransferenciaProduto` é usado quando a qualidade destina quantidade para refugo ou reclassificação. Nesse caso, a regra cria saída do produto/lote original e entrada no produto/derivacao/lote destino. Também preserva a ligação de rastreabilidade em campo customizado, para que o rastreamento consiga reconstruir a cadeia mesmo após transferência/reclassificação.

6.2.5 Análise do campo `acaoBotao` nos webservices de lote

O campo `acaoBotao` faz parte do contrato entre o SIGMA e os webservices customizados do Senior/Sapiens nos fluxos de lote. Ele não é apenas uma informação visual da tela. No LSP, esse campo é lido e usado para atualizar a situação customizada do lote em `E210DLS.USU_SITLOT`.

No Python, dois pontos montam esse campo:

Origem Python	Serviço enviado	Valor de <code>acaoBotao</code>	Significado operacional
<code>setores/qualidade/views/cor</code>	<code>MovimentarEstoque</code> ou <code>TransferenciaProduto</code>	A	Liberação normal, refugo ou reclassificação a partir da consulta/liberação de lote.

O envio dos registros de qualidade é centralizado em `consulta_lote.py`. O campo `acaoBotao=A` identifica a destinação final registrada pela qualidade. Antes da reunião, a tela `liberar_lotes.py` também pode mover o lote pendente para o depósito de Área Vermelha com `acaoBotao=V`; essa transferência física usa a transação interna padrão da filial e deixa a análise/destinação final para a reunião.

No `WebService Apontamento/MovimentarEstoque.lsp`, o JSON recebido é lido e o valor de `acaoBotao` é usado para gravar a situação do lote em `E210DLS.USU_SITLOT` depois da movimentação, quando não há erro de webservice.

No `WebService Apontamento/TransferenciaProduto.lsp`, o mesmo campo também é lido do JSON e usado para gravar `E210DLS.USU_SITLOT`, considerando o lote original e o lote transferido/reclassificado.

Conclusão técnica: o campo `acaoBotao` não deve ser removido do payload sem ajuste correspondente nos LSP, pois ele continua fazendo parte do contrato de situação do lote enviado ao Senior.

Como o retorno é tratado:

- O código procura tag `<waRetorno>`.
- Quando o conteúdo é JSON:
 - sucesso se `message == "OK"` ou `status == "OK"`.
- Quando não é JSON:
 - sucesso se texto contém `Processado com sucesso` ou variante.
- Falhas retornam o registro para pendente (`status=0`) com log.

6.3 WMS XC API

O WMS recebe informações de lote e ajuste de estoque por API HTTP JSON. Diferente do ERP SOAP, aqui o envio é feito para endpoints REST/HTTP do WMS XC.

O SIGMA não envia todos os tipos de WMS do mesmo jeito. Quando a qualidade libera um lote novo para armazenamento o envio normalmente é `novo_lote`, que cria/recebe o lote no WMS. Quando na área vermelha apenas transforma um lote já existente manda `novo_lote` e ajuste no original avaliado. Quando o apontamento de componente consome um palete WMS, o envio normalmente é `ajuste`, que acerta saldo do palete/lote. Na tela Multi-OP (View 3, ver 8.2.1), cada baixa de bobina integrada no ERP também gera um `ajuste`, com a quantidade sendo o novo saldo real do lote — não um valor fixo.

Antes de enviar, a aplicação grava uma pendência local. Essa pendência é o controle de auditoria: informa o lote, paleta, quantidade, produto, derivacao, local WMS, status e log do retorno.

A fila respeita ordem por lote: antes de reservar uma pendência para envio, o sistema verifica se existe outra pendência mais antiga do mesmo (`empresa`, `lote`) ainda pendente ou em processamento; se existir, a mais nova fica bloqueada e não é enviada, mesmo que o worker automático ou o botão manual de envio tentem processá-la. Essa regra existe porque nem todo `ajuste` é idempotente: os ajustes de baixa de componente da View 3 carregam saldos diferentes a cada envio, então processar fora de ordem poderia regredir o saldo do WMS para um valor desatualizado. Na tela `/setores/qualidade/integracao-wms/`, uma pendência bloqueada por essa regra aparece como **Bloqueado (lote)**; uma pendência vinculada a uma reunião de área vermelha ainda aberta aparece como **Bloqueado (reunião)** — ambas em azul, mesma cor usada para bloqueio por ordem de fila nas demais telas de log do sistema.

Base:

WMS_XC_API_URL

Endpoints:

Tipo local	Endpoint	Objetivo
<code>novo_lote</code>	<code>rec_ska</code>	Receber/criar lote no WMS.
<code>ajuste</code>	<code>ajuste_estoque</code>	Ajustar saldo de lote/paleta existente.

Payload `novo_lote`:

Campo	Origem
WHSEID	Fixo WMWHSE1

Campo	Origem
STORERKEY	Fixo 00001
RECEIPTKEY	<origem>-<op>
TOLOC	Local WMS
SKU	Produto-derivacao
QTYRECEIVED	Quantidade
TOID	Paleta/lote
LOTTABLE01	Lote
USER	Usuário WMS do ambiente

Payload ajuste:

Campo	Origem
ARMAZEM	Fixo WMHSE1
LOTE	Lote da pendência
PALETE	Paleta da pendência
QTD_AJUSTADA	Quantidade
SKU	Produto-derivacao
USUARIO	Usuário WMS do ambiente

Retorno:

- Sucesso se HTTP 200 ou 201 e corpo não contem Erro.
- Falha se status diferente ou se corpo contem Erro.
- Falha retorna a pendência para status=0.

6.4 WMS via DBLINK Oracle

O DBLINK é usado quando a informação do WMS precisa ser consultada a partir do Oracle ERP. Isso evita que a aplicação abra uma conexão direta separada para algumas leituras específicas e aproveita a visão já exposta no ambiente Oracle.

Usos principais:

- Importação de paletes para USU_TPALWMS.
- Resolução de local WMS na área vermelha, isso é necessario para saber se o lote ja existe no WMS e se existe para qual local os lotes a partir dele devem ser gerados tambem.

Objeto principal:

```
wmhse1.v_XCLotxLocxId_Lottables@SQLDBLINK
```

6.5 Oracle Alchemy

O Alchemy e usado como apoio de qualidade e produção. Ele não e a base de estoque nem de OP; sua funcao e fornecer dados de análise/bobina que complementam a decisão operacional.

Usado em:

- Liberação de lote.
- Área vermelha.

- Rastreamento de lote.
- Importação de última bobina MP-III.

Tabela principal:

bobinas

Campos usados:

- `codmaquina`.
- `codbobina`.
- `flagproducao`.
- `observacao`.

6.6 MQTT

O MQTT é a integração de tempo real com recursos/dispositivos. No fluxo atual, ele é usado para receber valor de balança e atualizar as telas que dependem dessa leitura.

O sistema não depende apenas da tela aberta. O cliente MQTT roda no servidor e recebe mensagens continuamente. As telas que precisam ver balança em tempo real recebem atualizações por WebSocket.

Configuração:

- Broker: `MQTT_HOST`.
- Porta: `MQTT_PORT`, default 1883.
- Topico: `ooo/online`.
- Protocolo: MQTT v5.

Payload esperado:

- `recurso`.
- `balanca`.

Comportamento:

- Se `balanca` vem no payload, envia atualização via WebSocket para grupo `balanca_<recurso.id>`.
- O MQTT não grava mais histórico de estado do recurso.
- As paradas de máquina passam a ser registradas em `ParadaMaquina`, com uma parada em aberto identificada por `fim` vazio.

6.7 LDAP / Active Directory

O LDAP integra o SIGMA com o ambiente corporativo de usuários. Na prática, isso reduz a necessidade de manter senhas separadas dentro do SIGMA e permite que usuários usem a credencial de rede.

O grupo requerido `GP_Sigma` funciona como uma barreira de entrada: mesmo que o usuário exista no AD, ele precisa pertencer ao grupo autorizado para acessar o sistema.

Configuração:

- Servidor LDAPS: `ldaps://DC01.indaialpapel.com.br:636`.
- Dominio: `indaialpapel.com.br`.
- Grupo requerido: `GP_Sigma`.
- Certificado CA em `/opt/SIGMA/certs/ca_indaialpapel.pem`.

Comportamento:

- Backend LDAP vem antes do backend Django.
- Usuários LDAP podem ser criados automaticamente.
- Nome, sobrenome e e-mail são atualizados pelo AD.

6.8 SMTP

O SMTP é usado para notificações, principalmente no módulo de manutenção. Ele permite que abertura, atualização ou mudança de status de chamados e ordens de serviço gerem comunicação por e-mail para responsáveis e envolvidos.

Configuração:

- Host: outlook.office365.com.
- Porta: 587.
- TLS habilitado.
- Usado principalmente por manutenção para notificar chamados e ordens de serviço.

6.9 Retornos, status e tratamento de erro das integrações

Esta seção explica como o SIGMA decide se uma integração deu certo ou não. Isso é importante porque o sistema conversa com tecnologias diferentes: SOAP, JSON HTTP, Oracle e MQTT. Cada uma retorna erro de um jeito.

O princípio geral é: se a resposta externa não comprova sucesso, o SIGMA não marca o registro como integrado. Ele registra o erro no log e deixa o registro pendente para tentativa futura.

As filas usam status numerico para simplificar filtros e reprocessamento. O usuário ve isso nas telas como pendente, integrado ou processando. Na prática, a fila e o mecanismo que impede perda de informação. Quando uma ação precisa ser enviada ao ERP ou WMS, o SIGMA guarda um registro local. Esse registro pode estar pendente, processando, integrado ou em outro estado de controle.

Padrão geral:

- Sucesso grava `status=1`.
- Falha grava `status=0` e registra log.
- Em processamento usa `status=2`.
- Exclusão lógica ou bloqueio pode usar `status=3` em alguns modelos.

No SOAP ERP, o retorno pode vir com um JSON dentro de `<waRetorno>` ou com texto simples. Por isso, o SIGMA tenta interpretar das duas formas. Quando encontra um retorno OK, marca sucesso. Quando não encontra, considera falha. O sucesso pode ser indicado por `message == "OK"`, `status == "OK"` ou por texto equivalente a processamento concluído.

No WMS, o HTTP pode retornar 200 ou 201 mesmo quando o corpo indica erro. Por isso, o SIGMA não olha apenas o status HTTP; ele também verifica se o corpo contem a palavra **Erro**. Um retorno HTTP tecnicamente bem sucedido significa apenas que o WMS respondeu; não significa, sozinho, que o lote foi recebido ou que o ajuste foi aceito pela regra do WMS.

Nas consultas Oracle, a falha normalmente impede a continuidade da operação, porque o Oracle é fonte de verdade para saldo, lote, OP e movimentos. Em tela, a falha aparece como mensagem ao usuário. Em worker, o erro fica no status do serviço ou no log da fila.

Para reduzir falhas por texto mal decodificado, o SIGMA usa `producao.utils.codificacao` nos pontos de integração. O módulo fornece:

- `safe_decode`: decodifica payload recebido como bytes, usado principalmente no MQTT.
- `get_response_text`: lê respostas HTTP de SOAP/WMS/contadores web respeitando a codificação detectada ou fazendo fallback seguro.
- `safe_str`: transforma exceções e mensagens em texto seguro antes de salvar log de fila, status de service ou erro operacional.

Esse utilitário é usado nos fluxos MQTT, SOAP ERP, WMS, consulta/liberação de lotes, estorno de componentes, logs de apontamento, logs de componentes, importação de números de bobina e status dos services. A finalidade não é mudar a regra de negócio, mas preservar o retorno real recebido das integrações e impedir que um problema de codificação esconda a causa operacional da falha.

Quando a falha acontece durante uma ação de tela, o usuário recebe uma mensagem de erro e o registro não deve ser tratado como concluído. Quando a falha acontece em uma fila, o registro volta para pendente ou permanece em estado controlado para nova tentativa, conforme o ponto em que a execução parou.

Esse comportamento é comum aos fluxos de apontamento, componente, estorno, liberação de lote e WMS: o SIGMA só encerra a pendência quando existe retorno externo suficiente para considerar a operação concluída. Caso contrario, preserva a rastreabilidade local para suporte, correcao ou reenvio.

7. Operação em background, filas e monitoramento

Os workers são rotinas automáticas que rodam junto da aplicação. Eles existem porque muitas tarefas não devem depender de um usuário ficar esperando na tela. Enviar uma fila ao ERP, importar paletes do WMS, limpar sequenciamento ou recalcular planejado OEE são operações que precisam acontecer em segundo plano.

O ponto central é que o SIGMA não usa um agendador externo separado para esses workers. Eles sobem dentro do próprio processo Django/Daphne, controlados pelo app `accounts`. Para evitar que dois processos executem a mesma rotina ao

mesmo tempo, o sistema usa uma trava no PostgreSQL.

7.1 Inicialização

Quando a aplicação sobe, ela tenta iniciar os workers. Antes de iniciar, tenta adquirir uma trava no PostgreSQL. Se a trava já estiver em uso, aquele processo não sobe os workers. Isso protege contra duplicidade quando há mais de um processo Django ativo.

A trava usada é um advisory lock do PostgreSQL. Esse tipo de trava não bloqueia uma tabela nem um registro específico; ele funciona como um marcador lógico mantido pela conexão com o banco. O SIGMA usa esse marcador para responder a uma pergunta simples: “já existe outro processo autorizado a rodar os workers?”. Se a resposta for sim, o processo atual continua servindo a aplicação, mas não inicia as rotinas automáticas.

A chave 20260430 é o número que identifica essa trava dentro do PostgreSQL. Ela fica definida no código como `_CHAVE_TRAVA_WORKERS` em `accounts/apps.py` e é passada para `pg_try_advisory_lock`. Enquanto uma conexão mantém essa chave travada, outra tentativa com a mesma chave não consegue assumir os workers. Quando o processo encerra e a conexão é fechada, o PostgreSQL libera a trava automaticamente.

Regras:

- Em comandos Django, workers só sobem no `runserver`.
- Em produção, sobem quando o processo ASGI inicia.
- Um `pg_try_advisory_lock` no PostgreSQL evita que mais de um processo suba workers.
- A chave da trava global dos workers é 20260430.

7.2 Supervisor

O supervisor é uma thread permanente que acorda a cada 30 segundos. Ele verifica se os workers estão ativos, tenta reiniciar quando necessário e monitora ciclos travados. Essa checagem é importante porque algumas integrações externas podem travar, demorar ou falhar sem responder corretamente.

Thread:

SupervisorWorkersSigma

Intervalo:

30 segundos

Responsabilidades:

- Iniciar ou reiniciar workers.
- Monitorar services travados.
- Fechar conexões antigas de banco.

7.3 Tabela de workers

Cada worker tem uma responsabilidade diferente. Alguns rodam por intervalo fixo, como importação de bobinas ou limpeza de sequenciamento. Outros são disparados sob demanda, quando existe fila a processar.

Código	Nome	Intervalo	Limite ciclo	Finalidade
<code>envia_pendencias</code>	Envio automático de pendências	300s	60s	Dispara filas ERP/WMS/lotes/estornos/temp
<code>fila_logs_apontame</code>	Fila Log Apontamentos	sob demanda	1800s	Envia apontamentos OP ao ERP.
<code>fila_tempos_erp</code>	Fila Log Tempos ERP	sob demanda	1800s	Envia um pacote de produção/paradas ao ERP por chave de OP.
<code>fila_log_apontamer</code>	Fila Log Apontamento Componentes	sob demanda	1800s	Envia componentes ao ERP.

Código	Nome	Intervalo	Limite ciclo	Finalidade
<code>fila_baixa_compone</code>	Fila Baixa Componentes	sob demanda	1800s	Envia ao ERP as baixas de bobina geradas pela tela Multi-OP (View 3), via TratarBaixa.
<code>fila_estorno_comp</code>	Fila Estorno Componentes	sob demanda	1800s	Envia estornos ao ERP.
<code>fila_wms_integracc</code>	Fila WMS Integrações	sob demanda	1800s	Envia pendências WMS.
<code>fila_consulta_lote</code>	Fila Consulta de Lotes	sob demanda	1800s	Envia LiberacaoLote ao ERP.
<code>importa_numbobinas</code>	Importação de números de bobinas	60s	50s	Atualiza bobina atual dos recursos.
<code>importa_palete</code>	Importação de paletes WMS	3600s	180s	Solicita a sincronização WMS para USU_TPALWMS.
<code>sincroniza_ops_enc</code>	Sincronização de OPs encerradas	300s	60s	Para OPs S, F ou C no ERP, fecha a parada aberta no fim do período, congela a justificativa em andamento, encerra a OP ativa e remove o sequenciamento.
<code>consolida_tempos_erp</code>	Consolidação de tempos ERP	21600s	60s	Executa às 00h15, 06h15, 12h15 e 18h15; consolida produção e paradas desde o último corte gerado, mesmo que ciclos anteriores não tenham rodado.
<code>oeo_planejado</code>	Recalcula Planejado OEE	600s	180s	Consolida planejado do dia e do dia anterior.
MQTT	MQTTLoopThread	contínuo	n/a	Recebe leituras MQTT e atualiza balança em tempo real.
<code>coleta_telemetria</code>	Coleta de Telemetria	contínuo	timeout HTTP máximo + 10s	Coordena coletas HTTP posicionais por recurso em pool limitado.

7.3.1 Coleta de Telemetria

O coletor HTTP roda dentro da arquitetura existente do SIGMA; não usa management command, agendador externo nem processo separado. O bootstrap em `accounts/apps.py` usa a mesma trava global `pg_try_advisory_lock` dos demais workers, o supervisor garante início idempotente e reinicia a coordenadora se ela parar.

Há uma única thread coordenadora, `CoordenadorColetaTelemetria`. Ela carrega em memória as configurações ativas e os sensores ativos por recurso, monta a fila e delega uma coleta única por recurso a um `ThreadPoolExecutor`. O limite de coletas HTTP paralelas é definido pela variável `TELEMETRIA_MAX_WORKERS` e exibido no painel de status; cada recurso permanece no conjunto de execução até sua tarefa terminar, portanto não há duas coletas simultâneas para a mesma máquina.

O coordenador é orientado a eventos, sem polling fixo. Ele acorda imediatamente quando uma coleta termina, quando a configuração de um recurso é alterada ou quando recebe encerramento. Sem evento, aguarda até a próxima coleta agendada ou até o prazo de segurança de uma tarefa em execução. Depois de vencido esse prazo, atua como vigia até que o supervisor trate uma tarefa travada.

O máximo de coletas HTTP paralelas é definido pela variável de ambiente `TELEMETRIA_MAX_WORKERS`; o valor efetivo aparece ao expandir o serviço no painel de status. A margem de monitoramento do ciclo é definida por `TELEMETRIA_MARGEM_CICLO_SEGUNDOS`.

Cada recurso possui seus próprios campos de pausa após sucesso e espera após erro. Depois de uma coleta bem-sucedida, ele volta ao fim da fila somente após sua pausa. Depois de erro, volta após sua espera de erro. Uma máquina lenta ou sem resposta fica limitada pelo timeout daquele recurso e não bloqueia as demais posições livres do pool.

O coletor aceita resposta simulada nos testes e não depende de uma estrutura HTML fixa. Em execução, ele extrai textos na ordem em que aparecem no HTML, aplica a posição cadastrada e converte cada valor: vírgula decimal é aceita, booleanos aceitam `1/0`, `true/false`, `sim/não`, `yes/no` e `on/off`; inteiros, decimais, texto e valores nulos são tratados conforme o tipo do sensor. Resposta vazia, posição ausente, HTML sem valores, valor inválido, timeout e erro HTTP encerram a coleta como falha. O HTML não é salvo: somente o JSON interpretado com o código fixo do sensor como chave é persistido.

Depois de interpretar uma resposta HTTP com sucesso, o coletor mantém duas referências independentes por recurso: o **último snapshot coletado**, atualizado em toda resposta interpretada, e a **última leitura salva**, usada somente para decidir a persistência. A avaliação da parada automática usa o snapshot atual, mesmo quando a regra de gravação decide não criar uma `LeituraTelemetria`.

Nas primeiras coletas válidas do recurso, a leitura completa é salva. Nas próximas, o coletor compara o JSON completo com a última leitura salva mantida no cache do coordenador. Em qualquer tipo, o vínculo precisa estar marcado para monitorar variação para poder disparar uma nova gravação. Para decimal e inteiro, a mudança também precisa atingir a tolerância absoluta ou percentual; quando o valor anterior é zero, a mudança é tratada sem divisão por zero. Booleano e texto não configuram tolerância: quando monitorados, qualquer mudança dispara gravação. Se nenhum sensor monitorado disparar, não é criada nova leitura. Não há leitura parcial.

A regra de parada é avaliada em toda coleta HTTP interpretada com sucesso e sua falha é tratada separadamente da gravação de telemetria: um erro na regra não impede a leitura, e um erro ao gravar a leitura não impede a avaliação que já ocorreu. Falha HTTP, resposta inválida, sensor ausente ou valor incompatível não representam parada; o avaliador retorna **INDETERMINADO**, não abre nem fecha parada e reinicia a contagem de estabilidade.

`RegraParadaRecurso` é única por recurso e possui `ativa` e uma árvore JSONB validada. Cada folha referencia o código fixo de um sensor vinculado e ativo, uma comparação (`igual`, `diferente`, `maior`, `maior ou igual`, `menor` ou `menor ou igual`) e o valor esperado. Os nós de grupo usam `E`, `OU` ou `NÃO`, podem ser aninhados e precisam conter itens; `NÃO` possui exatamente um item. A validação confere a estrutura, o vínculo do sensor e a compatibilidade entre comparação, valor e tipo do sensor. A árvore é somente dados: não há `eval`, SQL, Python nem execução de texto armazenado. Resultado verdadeiro significa **PARADO**; falso significa **FUNZIONANDO**.

O campo **Tempo Parada Automática** do recurso é um intervalo não negativo e é o período mínimo de estabilidade antes de qualquer abertura, reabertura ou fechamento automático. Se o sinal oscilar, a coleta falhar, o período produtivo mudar ou o operador alterar uma parada manualmente, a contagem recomeça. Depois de uma parada fechada manualmente, o sinal ainda parado precisa permanecer estável desde o fim dessa parada mais o tempo configurado para que uma nova parada seja aberta. No sentido contrário, a parada só pode ser fechada depois de permanecer aberta pelo menos esse tempo. A abertura usa o momento da ação automática, sem retroceder o horário.

Ao indicar **PARADO**, a rotina reconsulta e trava os registros necessários dentro de transação. Ela exige que o recurso permita apontamento de parada, que exista ao menos um `LogTrocaOPAtiva` aberto e que não haja outra parada aberta; então cria uma única `ParadaMaquina` do tipo **Sinal**, vinculada a todos os períodos abertos do recurso. Ao indicar **FUNZIONANDO**, ela pode fechar a parada aberta atual, manual ou por sinal, sem criar outra. O fechamento preserva o operador, usuário e

data/hora da parada, ajusta somente o fim e, quando houver justificativas, congela a última: recalcula seu parcial depois da soma das anteriores, grava o tempo restante exato da parada e atualiza a data/hora dessa justificativa. As notificações `parada_update` são emitidas pelo fluxo já existente somente após o commit da transação.

O cache de configuração é carregado ao iniciar o coletor e é atualizado no processo quando se salva configuração, sensor, vínculo ou regra de parada do recurso. A primeira comparação de cada recurso busca a última leitura no banco; depois disso, a última leitura salva e o snapshot atual ficam em memória enquanto a coordenadora estiver ativa. O estado de estabilidade da regra é reiniciado quando a configuração do recurso muda. Ao reiniciar o processo, a primeira coleta volta a consultar a última leitura persistida.

Em toda coleta bem-sucedida, mesmo sem nova leitura, o status em memória é atualizado. Em falha, a configuração do recurso recebe a mensagem em `log` e o horário em `ultima_coleta_em`, cujo significado é **data/hora do último erro**. Não há atualização dessa data nem do log em caso de sucesso; as leituras persistidas são a referência dos sucessos.

No painel **Status (Services)**, Coleta de Telemetria aparece como serviço contínuo e pode ser expandida como o envio automático de pendências. A expansão lista cada recurso ativo com URL, timeout, pausas, situação (Coletando, Na fila ou Aguardando), próxima coleta, duração, último erro e data/hora do último erro. A duração mede toda a tarefa do recurso no pool: leitura de configuração e última leitura no cache, requisição HTTP, interpretação, avaliação independente da parada automática, comparação e eventual gravação. Durante uma tarefa em andamento, mostra o tempo já decorrido; ao terminar, mostra a duração final daquele ciclo. A parada automática é executada pela própria tarefa do recurso no pool existente; não cria worker, thread coordenadora, fila ou management command adicional.

7.3.2 Escalabilidade das consultas de histórico

As telas de histórico usam paginação no banco. A página corrente carrega somente os itens exibidos e suas relações necessárias; não percorre todo o histórico para montar a grade.

Tela	Paginação e índice de suporte
Logs Apontamentos	20 registros por página; índice por empresa/ID na entrada nativa e índice da fila por status e chave da OP. O bloqueio de envio é verificado por EXISTS somente nos registros da página.
Log Tempo Produção	14 períodos por página; índice por horário da troca e ID. Paradas e justificativas são consultadas apenas para os períodos exibidos.
Log Tempos ERP	20 pacotes por página; índice por fim do corte e ID. Os itens são carregados somente para os pacotes da página.
Logs Componentes	20 registros por página; índice por empresa/ID na entrada nativa e índice da fila por status e chave da OP.
Baixa Componentes	20 registros por página; índice por empresa/ID na entrada nativa (<code>idx_baixa_comp_codemp_id</code>) e índice da fila por status e chave da OP (<code>idx_baixa_comp_fila_chave</code>). O botão de envio individual e o bloqueio visual por linha usam EXISTS para checar, sem carregar os registros, se já existe outra baixa do mesmo lote em processamento ou pendente antes dela.
Estorno Componentes	20 registros por página; a limpeza de retenção não é executada no carregamento da tela. O worker <code>envia_pendencias</code> remove estornos com mais de 60 dias e usa índice por status/data.
Consulta de Lotes	20 grupos por página; os grupos são paginados no banco por empresa, bobina, lote, produto e derivação. Os registros, reuniões e participantes são carregados apenas para os grupos da página.
Chamados de Manutenção	20 registros por página; não possuem índices personalizados para filtros de responsável, observador, recurso, status ou categoria.
Ordens de Serviço	20 registros por página; não possuem índices personalizados para filtros de responsável, recurso ou status.

O Paginador executa `COUNT(*)` para informar o total de páginas. Isso não traz os registros para a aplicação; é uma contagem no banco. Os índices acima reduzem o custo dos filtros e ordenações mais usados. Buscas livres com `icontains` em texto

longo continuam sendo mais caras por natureza e devem ser usadas para localizar casos específicos, não como carga padrão da tela.

7.4 Timeout e reprocessamento

O tratamento de timeout existe para impedir que um registro fique preso eternamente como “processando”. Quando uma fila reserva um registro, ele muda para o status de processamento definido para a própria fila. Se o processo cair, travar ou for interrompido, esse registro poderia ficar bloqueado. O worker de envio de pendências e o supervisor tratam esse caso liberando registros antigos de volta para pendente.

Padrão das filas:

- `status=0`: pendente.
- `status=1`: integrado.
- Cada fila mantém os demais status conforme sua própria tabela. Em PacoteTempoERP, `status=2` é erro e `status=3` é processando.

Reserva:

- Antes de enviar, cada fila tenta assumir um lock de execução em memória, como `PROCESSAMENTO_LOGS_LOCK`, `PROCESSAMENTO_APONTAMENTO_COMPONENTES_LOCK`, `PROCESSAMENTO_WMS_LOCK`, `PROCESSAMENTO_LOTES_LOCK` ou `PROCESSAMENTO_ESTORNOS_LOCK`. Esse lock evita que duas threads da mesma fila rodem ao mesmo tempo dentro do mesmo processo Django.
- Depois disso, a fila faz a reserva no banco usando uma transação com `select_for_update(skip_locked=True)`. Esse lock transacional protege os registros no momento da seleção, evitando que duas execuções reservem o mesmo item ao mesmo tempo.
- Os registros selecionados são marcados com o status de processamento da fila. Esse status é a reserva persistente: indica que o item já foi capturado e não deve ser pego por outra execução enquanto estiver nessa condição.
- O lock transacional dura apenas durante a reserva no banco. O que permanece depois da transação é o status de processamento e o lock de execução da thread enquanto o envio estiver rodando.

Timeout:

O controle de timeout tem dois níveis. O primeiro é o timeout da chamada externa, usado quando uma fila está aguardando resposta de HTTP, SOAP ou WMS. Esse prazo normalmente é de 180 segundos. Enquanto a última atividade registrada do service ainda está dentro desse prazo, o ciclo não é tratado como travado, mesmo que ele ainda esteja em andamento.

O segundo nível é o limite do ciclo do service. Esse limite protege o worker contra execução presa sem progresso. A cada verificação, o supervisor compara o tempo de execução do ciclo com o limite configurado e também verifica se existe uma atividade externa recente ainda dentro do timeout do webservice. Se ainda houver chamada válida em andamento, o supervisor aguarda. Se o ciclo passou do limite e não existe atividade válida dentro do prazo, `matar_service_travado` solicita a interrupção da thread.

Depois da interrupção, o próximo ciclo verifica os registros que ficaram no status de processamento. Quando o lock da fila está livre e a reserva já passou do tempo seguro, esses registros voltam para `status=0` para novo processamento.

7.5 Manual operacional do painel de Status (Services)

O painel de status dos services não é apenas uma tela técnica. Ele é a principal visão operacional para entender se os workers estão rodando, se alguma fila esta presa, se existem pendências elegíveis para envio e se o processo Django ainda possui telemetria ativa dos services.

Arquitetura de execução:

- `accounts/apps.py` é o ponto de bootstrap dos workers. Ele inicia as rotinas automáticas somente em processo elegível e usa `pg_try_advisory_lock` para evitar que dois processos executem os mesmos workers ao mesmo tempo.
- `SupervisorWorkersSigma` roda em thread daemon e tenta chamar novamente os `start_*` principais. Cada worker decide internamente se precisa subir ou se já existe uma instancia ativa.
- `producao/services/status.py` é o registro operacional em memória do processo. Ele não substitui log persistente; serve para telemetria em tempo real, ciclo atual, duracao, erro, thread ativa e decisão de timeout.
- `MonitorConexoesPostgres` verifica a ocupação do PostgreSQL e a fila do PgBouncer sem manter conexão própria aberta entre as verificações. Ao cruzar 100 conexões de cliente, grava no journal uma única captura das sessões; quando o total volta a 70 ou menos, registra a normalização e fica pronto para uma nova captura. Se houver cliente aguardando vaga no PgBouncer, registra uma ocorrência da fila e só rearma quando ela zera.
- A faixa **Servidor** da tela mostra as conexões PostgreSQL atuais sobre a capacidade normal disponível e separa quantas

estão em execução. A capacidade normal exclui as conexões reservadas para superusuário.

- Quando `PORTAL_BASE_URL` usa HTTPS, a faixa **Servidor** consulta o certificado apresentado pelo domínio configurado na porta 443 e mostra a data de expiração e os dias restantes. Fica amarela abaixo de 30 dias, vermelha abaixo de 7 dias e informa indisponibilidade se a consulta TLS falhar. Em instância sem HTTPS configurado, esse indicador não é exibido.
- Services principais mantém loop próprio. As filas de integração são threads filhas assíncronas, disparadas quando existe pendência elegível.
- A importação de números de bobina consulta a MP-III no Oracle Alchemy pela thread principal e cria threads filhas apenas para consultar em paralelo os contadores web MP-IV e MP-V, cada uma com timeout de 30 segundos. O ciclo completo tem limite de 50 segundos.
- A importação de paletes WMS chama imediatamente ao iniciar e depois a cada hora o Webservice `ImportaWMS`, com a ação `IMPORTAR-PALETES` e as chaves `chave` e `valor` (vazias nesta ação). O ERP executa o `MERGE` em `USU_TPALWMS` e devolve a quantidade de registros inseridos ou atualizados. O timeout SOAP e o limite de ciclo são de 180 segundos; um lock próprio impede duas importações simultâneas. Ela é um scheduler independente, não uma fila filha do Envio automático de pendências.
- A Sincronização de OPs encerradas consulta as OPs locais abertas no ERP. Quando encontra situação `S`, `F` ou `C`, encerra o período produtivo no instante atual e remove a OP do sequenciamento local. Se a parada física vinculada não possuir outro período ainda aberto, também grava seu fim nesse mesmo instante e congela a última justificativa para cobrir a parada; caso exista outro período aberto associado, a parada permanece aberta. Após confirmar a transação, envia `refresh_page` pelo WebSocket do recurso para a tela refletir a OP encerrada.
- Envio automático de pendências, Sincronização de OPs encerradas e Consolidação de tempos ERP possuem limite de ciclo de 60 segundos. Recalcula Planejado OEE possui limite de 3 minutos.

Orquestrador de integrações:

- O Envio automático de pendências coordena o disparo das filas. Ele não processa os registros diretamente e não espera a fila filha terminar; ele verifica locks, trata interrupções por timeout e dispara nova thread quando aplicável.
- Antes de disparar novas filas, o orquestrador chama o controle de timeout das filas e services de apoio monitorados: Consulta de Lotes, Estorno Componentes, Logs Apontamentos, Log Apontamento Componentes, Baixa Componentes, WMS Integrações, Importação de números de bobinas, Envio automático de pendências, Sincronização de OPs encerradas, Consolidação de tempos ERP e Recalcula Planejado OEE. A importação de paletes mantém monitoramento próprio no seu scheduler. A Fila Baixa Componentes também mantém o lock próprio (`PROCESSAMENTO_BAIIXAS_COMPONENTES_LOCK`) e devolve para pendente registros travados em processamento.
- Logs Apontamentos, Log Tempos ERP, Log Apontamento Componentes, Baixa Componentes, Estorno Componentes, WMS Integrações e Consulta de Lotes são disparadas automaticamente quando existem pendências elegíveis e o lock da fila esta livre. A fila de tempos reserva o pacote, mantém somente uma chave de OP em processamento e registra o retorno no próprio pacote.
- Estorno Componentes também pode ser disparado pela tela própria ou pelo fluxo de estorno/redução, mas segue o mesmo mecanismo de registro, ciclo e timeout das demais filas.
- A Fila Baixa Componentes é disparada pelo Envio automático de pendências quando encontra registros pendentes e também abre uma thread própria (`disparar_envio_baixas_componentes`) imediatamente após cada apontamento bem-sucedido ou repesagem na tela Multi-OP. Ela ainda pode ser disparada manualmente pela tela `/producao/logs-baixa-componentes/`, individual ou para todas as pendentes. A fila reserva por lote (CodLot): só processa a próxima baixa do mesmo lote depois que a anterior for integrada, preservando a ordem cronológica do consumo daquela bobina. A tabela física é `producao.baixa_componentes`; o índice `idx_baixa_comp_fila_lote` em (`status`, `codlot`, `id`), criado pela migration `producao.0037_baixacomponente_fila_codlot`, atende a busca ordenada de pendências por status e lote usada nessa reserva.

Reserva e status dos registros:

- `status=0`: não integrado/pendente, disponível para reserva e envio.
- `status=1`: integrado com sucesso.
- O status de processamento depende da fila. Em Log Tempos ERP, `status=3` significa processando e `status=2` significa erro.
- `status=3`: excluído em Apontamentos e Componentes; não é status operacional comum a todas as filas do painel.
- A reserva é feita no banco antes do envio. Nas filas que usam concorrência, esse controle reduz captura duplicada entre execuções concorrentes.
- Registros no status de processamento voltam para pendente em dois cenários seguros: depois de uma interrupção por timeout, ou quando o lock da fila esta livre e a reserva ficou parada por mais tempo que o timeout webservice daquela fila.

Controle de ciclo e timeout:

- Cada service registrado informa intervalo e, quando aplicável, limite máximo de ciclo. O registry calcula ciclo em andamento, próximo ciclo, duração e condição de travamento.
- Durante cada chamada externa, a fila marca uma atividade válida até o timeout configurado do próprio webservice. Enquanto esse prazo estiver vigente, o ciclo não é tratado como travado.
- Se o ciclo ultrapassar o limite e não houver chamada externa ainda dentro do prazo de retorno, o orquestrador solicita interrupção da thread registrada.
- A interrupção é sinalizada no registry. No ciclo seguinte, quando o lock da fila não estiver mais preso, os registros reservados daquela fila são devolvidos para pendente e reenviados.
- Se uma thread filha morrer sem registrar timeout, o orquestrador também libera reservas antigas: com lock livre, registros no status de processamento há mais tempo que o timeout webservice da fila voltam para status=0.

Leitura das filas de integração:

- Pendentes são registros ainda não reservados para envio.
- Elegíveis consideram regras de negócio da fila, como reunião fechada para WMS e Consulta de Lotes.
- Bloqueados são pendências que existem, mas não podem ser enviadas naquele momento por regra de negócio.
- Processando são registros reservados por uma thread ativa ou por uma execução anterior que ainda não foi liberada pelo timeout ou pela regra de lock livre baseada no timeout webservice.
- Timeout webservice mostra quanto tempo cada chamada externa daquela fila pode aguardar retorno.

Limites conhecidos do painel:

- O painel reflete o estado do processo Django que está executando os workers. Como o registry é em memória, reiniciar o processo limpa a telemetria exibida.
- A trava global dos workers depende do PostgreSQL. Se a aplicação não estiver usando PostgreSQL como banco principal, o bootstrap não adquire essa trava.
- O timeout de ciclo protege fila presa; o timeout de webservice protege chamada externa. São responsabilidades diferentes e aparecem separadas na tela.
- A coluna de erro exibe a última falha conhecida pelo registry, não o histórico completo de logs do registro integrado.

8. Fluxos de negócio

8.1 Sequenciamento de produção

O sequenciamento é o módulo que organiza as ordens de produção vindas do ERP em uma fila operacional por recurso. Ele não substitui o ERP como origem da OP; ele cria uma visão local de prioridade e distribuição para a fábrica trabalhar no curto prazo.

O usuário seleciona empresa e centro de recurso. O sistema consulta o ERP para listar OPs liberadas, em andamento ou aptas para produção, considerando roteiro, operação, produto, derivação, quantidade prevista, tempo previsto e saldo de estoque. A tela permite distribuir essas OPs entre recursos do centro e depois consolidar a ordem planejada no banco local.

O sequenciamento automático prioriza a necessidade de estoque: calcula $QTDEST - QTDRES - ESTMIN$, ordena primeiro as OPs com menor saldo disponível e usa o tempo de operação como desempate. Depois distribui a proposta entre os recursos com menor carga acumulada. Essa proposta só vira oficial quando o usuário consolida. Enquanto não consolida, a distribuição é apenas simulada na tela.

A consulta trabalha com produto e derivação da OP. `E900QD0.PROORI = 'S'` mantém as OPs com derivação definida; `E900QD0.QTDPRV` informa a quantidade prevista; e a descrição exibida reúne produto e derivação. Para cada par produto/derivação, o sistema soma todos os depósitos de `E210EST` e obtém estoque, reservado, estoque mínimo e máximo. Esses resultados ficam em cache durante a consulta para não repetir a busca do mesmo produto/derivação.

A tela ERP exibe Origem, OP, Produto, Derivação, Descrição, quantidade prevista, tempo, estoque, reservado, estoque mínimo e máximo. Estágio, sequência de roteiro e operação permanecem ocultos na tela porque são necessários para consolidar a sequência. A lista de recursos mostra OP | Origem | Produto | Derivação | Descrição. A exportação Excel segue essas colunas, formata o tempo como Xh Ymin e inclui Recurso Consolidado e Ordem Consolidada.

Tabela Oracle

Uso no sequenciamento

E900COP

Cabeçalho da OP: origem e número da OP.

Tabela Oracle	Uso no sequenciamento
E900QDO	Produto, derivação, quantidade prevista e filtro de origem do produto.
E90000P	Estágio, sequência, operação e tempo do roteiro.
E075PRO e E075DER	Descrições do produto e da derivação.
E725CRE e E083ORI	Centro de recurso e origem da OP.
E210EST	Estoque, reservado, estoque mínimo e máximo por depósito.

Fluxo:

1. Usuário acessa a tela de sequenciamento.
2. Sistema carrega empresas/centros conforme permissão e filial.
3. Ao consultar um centro, busca OPs no Oracle ERP.
4. Usuário distribui OPs nos recursos ou usa sequenciamento automático.
5. Consolidação apaga o sequenciamento anterior do centro e grava novo sequenciamento local.
6. Exportacao gera Excel com dados do ERP e recurso/ordem consolidada.

8.1.1 Calendário de OPs

O **Calendário de OPs**, no menu **Setores > PCP**, é uma visão de consulta do planejamento e da execução dos estágios das OPs no ERP. Cada evento representa um estágio da OP, identificado na linha por OP, produto-derivação e abreviatura do recurso (E725CRE.ABRCRE). As cores distinguem eventos que coincidem ou se encostam no período; não indicam situação da OP.

Cada ponta do evento usa a data real do estágio quando ela é válida (E900EOP.DTRINI para início e E900EOP.DTRFIM para fim); quando uma dessas datas não existe, somente aquela ponta usa a respectiva previsão (DTPINI ou DTPFIM). OPs finalizadas (F) são limitadas ao período entre o primeiro dia do mês anterior e o fim da visualização, evitando que a seleção traga todo o histórico do ERP.

Os filtros de máquina, origem, produto-derivação e situação são de múltipla seleção; quando algum valor está selecionado, o respectivo seletor fica azul. O filtro de produto-derivação preserva a distinção entre derivação nula e derivação composta somente por espaço, pois ambas existem no ERP. As situações disponíveis são: A andamento, L liberada, E explodida, R reabilitada e F finalizada; as quatro primeiras vêm selecionadas inicialmente. As linhas do calendário exibem o percentual, usam fonte ampliada e a área do calendário possui rolagem horizontal; abaixo do nome do mês, o rótulo informa que % é o percentual comprometido da OP.

Ao posicionar o mouse sobre qualquer parte do evento, o detalhe mostra situação, origem, estágio, máquina, período previsto e real, produto-derivação com suas descrições, quantidade prevista, quantidade já produzida, quantidade reservada e os pedidos reservados. A reserva é obtida em USU_TRESPED, filtrada por USU_CODEMP e USU_CODFIL da filial do usuário e por USU_NUMORP da OP; USU_QTDRES é somada por USU_NUMPED e no total comprometido.

O comprometimento é calculado somente quando o fim previsto do cabeçalho da OP (E900COP.DTPFIM) é igual ou posterior à data atual e o fim real (E900COP.DTRFIM), quando informado, ainda não passou. Para OP com fim previsto ou real passado, o calendário não mostra percentual nem dados de comprometimento no tooltip e não abre o modal. Ao clicar em uma OP elegível, o modal **Detalhamento do comprometimento** apresenta o estoque por depósito da E210EST, todas as OPs abertas de produção da E900QDO, todas as OPs abertas de consumo da E900CMO e as reservas por pedido da USU_TRESPED. Cada reserva também mostra a previsão de estoque do pedido em E120PED.USU_PRVEST, vinculada por empresa, filial e número do pedido. O estoque ignora os depósitos 01.25, AV.01, 05.14, 01.14, 01.23, 01.DV, C1.25, OP.25, P01.01, P01.02, P01.03 e 01.30. As tabelas de produção e consumo exibem origem, número, situação, fim previsto, quantidade prevista, realizada e pendente; as fórmulas do resumo identificam quais registros entram em cada intervalo pela data de fim prevista.

Para as origens configuradas em **Origens área vermelha** nos Parâmetros da Filial do usuário, o modal mostra três grupos de fórmulas por produto-derivação: **Comprometido até a data de corte**, **Comprometido após a data de corte** e **Com-**

prometido TOTAL. Em cada grupo são mostrados, em texto, $\text{Estoque} + \text{Produção pendente} = \text{Disponível}$, $\text{Consumo pendente} + \text{Reservas} = \text{Comprometido}$ e $\text{Disponível} - \text{Comprometido} = \text{Saldo}$, acompanhado do percentual que o comprometido representa do disponível. Nas demais origens, o comprometimento continua sendo exclusivamente a reserva da própria OP.

O preenchimento da faixa do evento mostra o percentual de **Comprometido após a data de corte**. Para as demais origens, ele é calculado como $\text{quantidade reservada da OP} / \text{quantidade prevista}$. Para produtos cujas origens estejam configuradas em **Origens área vermelha** nos Parâmetros da Filial, a data de corte é o fim previsto do cabeçalho da OP avaliada (E900COP.DTPFIM), mesmo que o evento esteja sendo desenhado por datas reais. Produção pendente é $\text{E900QDO.QTDPV} - \text{E900QDO.QTDRE1}$; consumo pendente é $\text{E900CMO.QTDPV} - \text{E900CMO.QTDUTI}$; ambos consideram somente OPs A, E, L e R, e nunca ficam negativos. O reservado é a soma de $\text{USU_TRESPED} \cdot \text{USU_QTDRES}$ por empresa, filial, produto e derivação, sem filtro por data.

Os três grupos não repetem OPs entre o cálculo até o corte e o cálculo posterior: **até a data de corte** parte do estoque atual, usa produção com E900COP.DTPFIM entre a data atual e a data de corte, inclusive, e consumo com DTPFIM menor ou igual à data de corte; **após a data de corte** parte do saldo obtido no grupo anterior e usa somente produção e consumo com DTPFIM maior que a data de corte, somando as reservas; e **TOTAL** usa o estoque atual, toda a produção e todo o consumo abertos, sem filtro de data, somando as reservas. A tabela continua exibindo todas as OPs abertas: ao lado do fim previsto, o triângulo vermelho aponta registros até o corte e o verde aponta registros posteriores; produção com fim anterior à data atual permanece listada, mas não recebe marcador nem entra no cálculo até o corte. Uma OP que começa na data de corte e termina depois dela entra no grupo posterior, pois a classificação considera somente seu fim previsto. A parte comprometida da barra mantém a cor da OP e o saldo restante usa a mesma cor misturada com 70% de branco. Para OP finalizada, o percentual não é exibido na linha.

Tabela Oracle	Uso no Calendário de OPs
E900COP	Cabeçalho e situação da OP.
E900EOP	Estágio e datas previstas/reais.
E90000P e E725CRE	Recurso do estágio e sua abreviatura.
E093ETG e E083ORI	Descrições de estágio e origem.
E900QDO, E075PRO e E075DER	Produto, derivação, quantidades e descrições.
USU_TRESPED	Pedidos e quantidade reservada para a OP.

8.2 Apontamentos de produção

O apontamento de OP é o fluxo usado para registrar produção realizada, refugo, lote, bobina, operador, recurso e horário de movimento. O SIGMA atua como camada operacional entre a tela de fábrica e o ERP: primeiro grava uma evidência local do apontamento e depois tenta integrar essa evidência ao ERP por webservice.

Esse desenho evita perda de informação quando o ERP ou a rede falha. Mesmo se o envio ao ERP não concluir, o registro local permanece com status pendente e pode ser reenviado automaticamente pelo worker ou manualmente pela tela de logs.

Ao entrar no apontamento por uma OP/código de barras, o SIGMA não usa uma tela fixa para todos os recursos. A tela base identifica o recurso da OP e consulta o campo `view_id` do cadastro do recurso. Esse campo aparece na aba OEE do recurso, mas hoje sua função operacional é selecionar a versão da tela de apontamento. Se o recurso estiver configurado com 1, o sistema carrega a versão `apontamentos_v1`; se estiver configurado com 2, carrega `apontamentos_v2`; se estiver configurado com 3, carrega `apontamentos_v3`, a versão Multi-OP descrita em 8.2.1. Os demais campos dessa aba permanecem cadastrados para uso futuro do módulo de OEE.

O envio ao webservice `Apontamentos` existe para efetivar a produção no ERP e, ao mesmo tempo, manter controle sobre a baixa de componentes. A regra Senior aponta a OP, evita duplicidade, controla início/fim quando necessário, desliga

temporariamente a baixa automática da OP e grava pendências de componentes para baixa posterior. Assim, o produto acabado é apontado no ERP sem perder o controle dos componentes que precisam ser consumidos com lote, saldo e rastreabilidade.

8.2.1 Apontamento Multi-OP (View 3, rebobinadeira)

A View 3 existe para um tipo de máquina que as versões 1 e 2 não conseguem representar bem: a rebobinadeira. Ela pode ter várias OPs acopladas ao mesmo tempo, porque o tempo de máquina precisa ser dividido entre elas, mas a produção nunca é apontada em mais de uma OP ao mesmo tempo — o operador pesa o lote de uma OP, tira da máquina, depois pesa o da próxima. Por isso a tela separa duas responsabilidades que nas versões anteriores eram uma só: **acoplamento de OP** (controla tempo, pode ter várias abertas) e **apontamento de produção** (sempre de uma OP por vez).

Acoplar uma OP não é livre: ela só entra se tiver ao menos um componente em comum — mesma combinação de produto e derivação — com todas as OPs já acopladas ao recurso. A razão é que o consumo de bobina de matéria-prima é **da máquina**, não da OP: todas as OPs acopladas compartilham as mesmas bobinas em consumo, então misturar OPs com componentes incompatíveis geraria baixa de matéria-prima errada para uma delas. A View 3 também não usa o botão de desacoplar OP das demais versões: o encerramento de uma OP acontece pelo próprio fluxo de alocação de bobina, sem depender de uma ação manual separada.

Consumo de bobina como fila. Cada bobina alocada ao recurso fica em `BobinaConsumoRecurso`, em um de dois estados: `EM_CONSUMO` (ativa, sendo debitada agora) ou `EM_FILA` (reserva, entra automaticamente quando uma bobina em consumo esgota). Quando o operador pesa um lote, o peso é dividido em fatias iguais pelo número de bobinas em `EM_CONSUMO` — se há duas bobinas em consumo, cada uma recebe metade do peso pesado, independente de qual OP foi apontada. Se uma fatia não cabe inteira na bobina correspondente, o restante daquela mesma fatia é emendado com bobinas da fila, na ordem, encadeando quantas forem necessárias; uma bobina da fila nunca cobre mais de uma fatia no mesmo apontamento, para que o rastreamento de qual bobina abasteceu qual pedaço do peso pesado não se perca.

Confirmação de consumo real antes de debitar. O cálculo de quanto cada bobina teria que fornecer é só uma estimativa teórica — uma bobina física pode render mais ou menos do que esse cálculo prevê. Por isso, o cálculo do rateio acontece em duas fases dentro da mesma transação: primeiro monta um plano teórico (sem gravar nada) e verifica quais bobinas esse plano zeraria; se houver alguma, a transação é abortada sem debitar nada (só houve leitura com `select_for_update`) e a tela volta pedindo confirmação, com a lista de bobinas pendentes guardada na sessão do recurso — o operador reenvia o mesmo apontamento junto com a resposta de cada bobina, sem precisar repetir peso, refugo ou OP selecionada. Para cada bobina pendente, o operador escolhe entre três opções: confirmar que consumiu tudo e a cadeia deve emendar com a próxima bobina da fila (só aparece quando o plano já previa essa próxima bobina); confirmar que consumiu tudo mas sem emendar, parando a cadeia ali — a próxima bobina da fila que seria usada fica intacta, sem debitar nada dela; ou informar que ainda resta um peso estimado, caso em que o sistema debita só a diferença entre o saldo teórico e o valor informado, nunca um valor negativo — se a bobina rendeu mais do que o previsto, nenhum débito é gerado para ela e ela sequer aparece na `BaixaComponente` gerada. Essa confirmação é a proteção de negócio para que uma OP, ao final, não apareça como tendo consumido mais bobinas do que era teoricamente esperado — o operador sempre tem a palavra final sobre o que realmente saiu da máquina. Uma bobina que zera de verdade (seja pelo plano teórico direto, seja por confirmação do operador) é removida da tabela de bobinas em consumo, e não apenas marcada como finalizada — o controle de repesagem, que é o único motivo para preservar histórico de um lote já retirado, é feito por uma tabela separada (`BaixaComponente`), então o registro de consumo em si não precisa continuar existindo depois de esgotado.

Geração de Apontamento e lote. Depois que o consumo de bobina é resolvido, o peso pesado é dividido pelas “Bobinas planejadas” da OP (`E900COP.USU_QTDBOB`) e cada parte gera um Apontamento com lote próprio, incrementado de `Empresa.loteatual` — mesma regra de geração de lote da v1. A diferença é que na v3 não existe número de bobina informado pelo operador: cada bobina cortada já é o próprio lote, então `Apontamento.bobina` fica sempre vazio. Quando a OP não tem `USU_QTDBOB` cadastrado no ERP, o sistema usa 1 como fallback em vez de bloquear o apontamento, para não deixar uma OP mal cadastrada impedir a produção. Refugo sem produção é permitido, condicionado ao parâmetro `ApontaRefugo` do recurso, do mesmo jeito que nas demais versões. Antes de gravar, o sistema confere no ERP se o estágio ainda existe na rota daquela OP — a mesma checagem feita na v1 — porque o estágio salvo no acoplamento pode ter ficado desatualizado se o roteiro for alterado depois.

Todo esse fluxo — rateio de consumo, confirmação do operador quando necessário e geração dos Apontamento — roda dentro de uma única transação: se algum Apontamento não puder ser gerado, o débito de bobina já calculado é desfeito, para nunca sobrar consumo debitado sem produção correspondente registrada.

Baixa de componente. Cada bobina de consumo efetivamente debitada em um apontamento gera um `BaixaComponente` local, um registro por bobina (não por lote de produção gerado, já que o consumo é da máquina e não de uma bobina de produto acabado específica). O `BaixaComponente` é enviado ao ERP pelo webservice `TratarBaixa` (ver 6.2.1), que grava a

pendência em `USU_TBxACMP` da mesma forma que os demais fluxos de apontamento — a execução física da baixa continua acontecendo pela regra orquestradora 230, fora do SIGMA.

Repesagem. Quando uma bobina que já teve consumo real é removida da máquina antes de zerar totalmente — porque o operador decidiu tirá-la, não porque ela esgotou —, o sistema gera um `BaixaComponente` com `repesagem=S`, e o lote fica bloqueado para nova alocação (em consumo ou fila) enquanto essa repesagem não for resolvida no ERP. A quantidade enviada nesse caso não é a estimativa local: `TratarBaixa` recalcula ela mesma como o saldo atual do lote no ERP menos 1, preservando saldo 1 para não repetir a repesagem em cada nova tentativa de baixa do mesmo lote. Esse bloqueio evita que o operador aloque de novo uma bobina cujo saldo real ainda não foi conferido pela balança/pesagem física, que é o motivo de existir a repesagem.

Ajuste automático no WMS. Cada baixa integrada com sucesso no ERP também gera um ajuste em `WMS_IntegraçãoOP` (`tipo_envio=ajuste`), para o WMS saber o novo saldo real do lote depois do consumo — o mesmo padrão usado pelo fluxo de apontamento de componente da v1/v2 (ver 8.4), mas com a quantidade calculada de forma diferente: em vez de sempre 0, aqui o valor é 1 quando a baixa é repesagem, 0 quando é consumo total, ou o saldo restante lido da própria `BobinaConsumoRecurso` no caso comum (por exemplo, uma bobina que tinha 1500 kg e debitou 1300 kg gera um ajuste de 200 kg). O paleta usado no ajuste é resolvido a partir do lote consumido, procurando em `USU_TPALWMS` por `USU_CODLOT`; sem paleta correspondente, o próprio lote é usado como identificador, igual ao que já ocorre no fluxo de área vermelha. Como o valor do ajuste muda a cada baixa do mesmo lote, cada baixa gera sempre um novo registro de ajuste, sem reaproveitar um pendente anterior — reaproveitar sobrescreveria um saldo que ainda não foi confirmado no WMS. Por esse mesmo motivo, a fila WMS (ver 6.3) só libera para envio a pendência mais antiga de cada lote: se houver uma pendência anterior do mesmo (`empresa`, `lote`) ainda pendente ou em processamento, as seguintes ficam bloqueadas até a anterior ser integrada — evitando que um reproprocessamento fora de ordem regrida o saldo já corrigido no WMS. Uma falha ao gerar esse ajuste não desfaz a baixa já confirmada no ERP; ela só fica registrada no log daquela baixa, sem bloquear as baixas seguintes.

A tela também interage com MQTT/WebSocket quando existe leitura de balança. A balança pode atualizar a interface em tempo real e ficar registrada junto do apontamento como evidência auxiliar.

O operador da tela de apontamento é validado contra o ERP em `E9060PE`. A OP ativa é controlada em `logs_troca_op_ativa`: ao trocar OP, a tela fecha o log aberto do recurso preenchendo `horario_saida` e cria um novo log com a OP decomposta em origem, op, estagio e seqrot, além de `horario_troca` e do código `id_operador`. A troca exige operador validado e é bloqueada se existir parada aberta ou pendente de justificativa. As telas `apontamentos_v1` e `apontamentos_v2` exigem exatamente uma OP ativa para qualquer ação de escrita; se o recurso tiver mais de uma, elas recusam a operação para não encerrar ou alterar todas por engano. Se uma troca alcançar uma parada física aberta, ela permanece a mesma e é vinculada ao período novo; cada período consolida somente sua própria parte temporal, sem criar parada artificial nem intervalo de segundos. O campo `ParadaMaquina.operador` guarda sempre o código do operador, não o nome. Os registros locais de `Apontamento` e `ApontamentoComponente` também gravam `usuario_id`, indicando o usuário Django autenticado que executou o apontamento na tela.

8.3 Tempos de produção, paradas e justificativas

Um **período produtivo** é o intervalo de permanência de uma OP em um recurso. Ele é o registro `LogTrocaOPAtiva`: começa em `horario_troca`, recebe a OP decomposta e permanece aberto até `horario_saida`. Em operação normal, há somente um período aberto por recurso. A primeira OP apenas cria o período; cada troca posterior fecha o período ativo e cria o período da nova OP.

Nas Views 0, 1 e 2, a lista de sequenciamento da tela de apontamento permite **desacoplar** a OP ativa. Desacoplar não exclui a OP do sequenciamento: somente encerra o período produtivo aberto, deixando o recurso sem OP ativa. A ação só é exibida e aceita quando não existe parada aberta nem parada fechada com justificativa incompleta. A mesma regra é validada no backend. A View 3 não mostra nem aceita esse botão: o encerramento de uma OP é feito pelo fluxo de alocação da própria tela, sem afetar as demais OPs do recurso. Nela, uma OP só pode ser acoplada quando houver pelo menos um componente comum — mesma combinação de produto e derivação — entre ela e todas as OPs que já estão acopladas ao recurso (ver 8.2.1 para o fluxo completo de acoplamento, consumo de bobina e baixa de componente dessa tela).

`ParadaMaquina` representa a parada física do recurso e guarda início, fim, código do operador, usuário que fez a última alteração, tipo (`Manual` ou `Sinal`) e data/hora de log. Cada parada se relaciona a um ou mais períodos produtivos do mesmo recurso. Uma parada aberta é vinculada a todos os períodos abertos do recurso; se uma OP for alocada enquanto a máquina estiver parada, o período novo também recebe o vínculo. Uma parada fechada lançada manualmente é vinculada a todos os períodos que cruzarem seu intervalo. Há no máximo uma parada aberta por recurso. Assim, um período pode conter nenhuma, uma ou várias paradas, e a tela Log Tempo Produção mostra a hierarquia período → parada → justificativas. A parada e suas justificativas são físicas e únicas, mesmo quando vinculadas a mais de um período. Na consolidação, cada período recebe somente a interseção da parada com seu próprio intervalo, sem antecipar o início do período nem alterar o início físico da parada.

No **Log Tempo Produção**, quem possui a permissão **Pode Alterar Paradas** pode corrigir os horários físicos de início e fim, com precisão de segundos. A alteração é única para a parada, mesmo quando ela aparece em mais de uma OP: ao fechar ou corrigir o intervalo, o SIGMA recompõe os vínculos pelos períodos que realmente o cruzam. A mesma reconciliação é executada quando a parada termina por telemetria, encerramento manual ou sincronização de OP. Por exemplo, se o fim for trazido para antes do começo da OP B, B deixa de contabilizar essa parada; se ainda houver interseção, conserva somente o trecho correspondente. Enquanto a parada estiver aberta, todos os períodos ainda abertos que a cruzam permanecem obrigatoriamente vinculados; um período já encerrado pode ser removido somente se não tiver participado do intervalo físico. Parada já fechada não pode ser reaberta por essa ação. As justificativas anteriores mantêm seus motivos e durações, seus inícios parciais são recalculados a partir do novo início e a última sequência recebe o saldo; a redução é recusada quando as sequências já fixas não couberem no novo intervalo. A exclusão administrativa usa o mesmo recálculo dos pacotes ERP locais que a alteração de horário.

Os motivos não são gravados diretamente na parada. A aba **Motivos de Parada** do recurso cadastra a abrangência local da combinação ERP CODEMP, CODGPM e CODMTV. `USU_T018GPM` fornece os grupos ativos; `USU_T018MVP` liga grupo e motivo; e `E018MTV` fornece o motivo e a descrição ativos. Ao justificar, o SIGMA só aceita motivo que esteja vinculado ao recurso e ainda ativo no ERP.

Cada `JustificativaParada` representa uma parte cronológica da parada e possui sequência, motivo, início parcial e tempo. A soma dos tempos de uma parada fechada deve ser exatamente igual à duração entre início e fim. Em parada aberta, a última justificativa fica sem tempo e representa o trecho ainda em andamento; ao encerrar a parada, esse último tempo é recalculado para fechar toda a duração. É possível dividir uma parada, por exemplo, em tempo de setup e tempo de quebra, sem perder a linha do tempo.

O bloqueio de apontamento ocorre por duas condições: parada aberta sempre bloqueia; parada fechada bloqueia enquanto as justificativas não cobrirem toda a duração. Nas telas que exibem justificativas, o operador deve justificar todas as pendências e, quando necessário, usar **Encerrar todas as paradas abertas** para informar o fim forçado. Essa regra também bloqueia troca de operador e troca/desacoplamento de OP enquanto houver pendência.

Paradas manuais exigem `Aponta Parada` no cadastro do recurso. A abertura é permitida quando o recurso possui `Permite Parada Manual` ou quando o usuário recebe a permissão **Pode Alterar Paradas**, diretamente ou por grupo. No apontamento, a abertura usa automaticamente o horário atual, o operador validado e o usuário logado, vinculando a parada física aos períodos abertos do recurso. No **Log Tempo Produção**, o usuário autorizado informa somente recurso, operador, início e, opcionalmente, fim: não seleciona OP nem período de referência. O SIGMA localiza e vincula automaticamente todos os períodos do recurso que cruzarem o intervalo. Os horários devem ser anteriores ao momento atual, o intervalo fechado deve respeitar o tempo mínimo configurado no recurso e não pode coincidir com outra parada; é obrigatório existir ao menos um período produtivo coincidente. Mesmo com uma parada aberta, pode registrar uma parada histórica já fechada se o intervalo não coincidir; uma segunda parada aberta continua bloqueada.

Todo save, exclusão ou alteração de vínculo de parada e justificativa publica, após o commit, o estado atualizado no WebSocket do recurso: se há parada aberta, quantas pendências existem e se o apontamento deve bloquear. Com isso, todas as abas abertas do mesmo recurso atualizam sem abrir WebSockets separados para cada tipo de evento.

Na versão `apontamentos_v1`, depois que um apontamento local é salvo com sucesso em `producao.apontamento`, a validação visual do operador é invalidada para exigir nova validação no próximo apontamento. Como essa tela aguarda alguns segundos antes de recarregar automaticamente, ela também marca no navegador que o operador foi limpo; se o usuário atualizar manualmente antes do contador terminar, a tela já volta pedindo novo ID de operador. Na versão `apontamentos_v2`, o operador permanece validado até troca manual do operador, troca de recurso ou nova regra específica da tela.

Quando o recurso está com `aponta_parada` e `exibir_jus` ativos ao mesmo tempo, uma `ParadaMaquina` aberta bloqueia o envio de apontamento; uma parada fechada também bloqueia enquanto a soma das justificativas for menor que sua duração. O modal lista as ocorrências e cria a primeira justificativa individual por parada, com um único operador validado em `E9060PE`, aplicado a todas as pendências. Ao lado do status de cada parada, o modal também mostra o tempo total decorrido: para parada já fechada, é um valor fixo calculado no momento da renderização; para parada ainda aberta, um contador no navegador recalcula esse tempo a cada segundo a partir do horário de início, sem precisar recarregar a página. Cada parada pode possuir várias `JustificativaParada`: cada sequência inicia em `parcial`, guarda o seu `CODMTV` e seu tempo; numa parada aberta a última sequência fica em andamento e é congelada quando a parada termina. Alterações posteriores são feitas no **Log Tempo Produção**, de acordo com a regra **Alteração de Justificativas** do recurso ou a permissão **Pode Alterar Paradas**. Salvar justificativas altera somente as respectivas `JustificativaParada`; o usuário, operador e data/hora da `ParadaMaquina` permanecem inalterados. Ao abrir, encerrar ou alterar os horários da parada, o sistema registra em `usuario` o usuário Django autenticado responsável pela ação. Cada motivo é revalidado no ERP pelo tipo vinculado e por `E018MTV.SITMTV='A'`.

A parada manual usa a mesma tabela e o mesmo WebSocket das demais paradas. Nas telas de apontamento, o botão abre

uma parada no recurso com início atual, tipo Manual, código do operador validado e usuário Django logado. Ele exige período produtivo aberto, operador validado, nenhuma parada aberta e autorização pelo parâmetro `permite_parada_manual` ou pela permissão `Pode Alterar Paradas`; `aponta_parada` é obrigatório em todos os casos. No Log Tempo Produção, o usuário autorizado informa recurso, código do operador, início e fim opcional. O sistema associa a mesma parada física a todos os períodos do recurso que cruzarem o intervalo, aplica o tempo mínimo do recurso e recusa sobreposição com outra parada. Toda inclusão, alteração ou exclusão de parada/justificativa notifica o grupo WebSocket do recurso após o commit, atualizando as abas abertas.

O submenu de apontamentos possui a tela Log Tempo Produção, acessível pelo atalho azul Log Tempos nas telas de apontamento quando há OP carregada. Cada registro principal representa um período produtivo controlado pela troca de OP: mostra a OP, o recurso, o código do operador, o início e o fechamento do período. A primeira coluna contém a seta de expansão da produção e das paradas; ao expandir, a tela apresenta as paradas vinculadas ao período e, dentro delas, as justificativas sequenciais. Sem `horario_saida`, o período aparece como Parado se existir parada aberta ou como em produção nos demais casos. A tela pagina 14 períodos por página. Uma parada aberta não pode perder vínculo nem ter período vinculado excluído: a relação é preservada até o fim físico da parada, inclusive se o período já tiver sido encerrado. Depois de fechada, a correção dos horários recompõe automaticamente os vínculos temporais; a exclusão administrativa pode remover um vínculo restante, e a parada física só é excluída quando não resta nenhum período associado. Ao corrigir ou excluir um trecho que já possui pacotes locais, os pacotes pendentes ou com erro daquele corte e dos cortes posteriores mantêm seus cortes e têm seus itens regenerados, sem o item de parada que deixou de existir. Pacotes integrados ou em processamento permanecem imutáveis no SIGMA e não retornam à fila; qualquer ajuste correspondente no ERP é manual. Pacotes anteriores ao trecho corrigido permanecem intactos.

8.3.1 Consolidação e envio de tempos ao ERP

O Log Tempos ERP transforma o histórico local em pacotes para o webservice `ApontamentoTempos`. Cada `PacoteTempoERP` pertence a um único período de `LogTrocaOPAtiva` e representa um intervalo real, guardado em `corte_inicio_real` e `corte_fim_real`. Os itens do pacote não repetem os dados da OP ou do recurso: eles guardam somente tipo (`PRODUCAO` ou `PARADA`), operador, motivo quando for parada, data/hora inicial e data/hora final.

O worker `consolida_tempos_erp` é agendado para 00h15, 06h15, 12h15 e 18h15. A agenda não obriga cortes físicos de seis horas: em cada execução, o início é o fim do último pacote do período e o término é o horário atual ou o fechamento do período. Assim, se a rotina ficar dois dias sem executar, ela gera o intervalo completo ainda não consolidado, sem criar cortes artificiais intermediários.

Para cada pacote, o SIGMA cria uma produção para todo o intervalo e inclui as paradas que se sobrepõem a ele. O motivo da parada é a justificativa que ocupou o maior tempo dentro daquele intervalo. Produção e parada são enviadas em listas separadas, mas compartilham a mesma chave de OP do pacote: empresa, origem, OP, estágio, roteiro e máquina.

Os horários enviados ao ERP são consistidos em minutos inteiros e gravados nos itens em horário local. Antes de criar o pacote, início e fim do corte têm segundos e microssegundos removidos; assim, um corte exibido como 12:11 gera Produção terminando em 12:11, sem avanço para 12:12. O payload usa sempre HH:MM:00. O ERP não aceita 00:00, portanto início ou fim exatamente à meia-noite seguem como 00:01.

As paradas são encaixadas entre esses limites sem sobreposição. No início, a primeira parada só pode começar um minuto após o início da Produção e após a última parada do pacote anterior, quando houver. No fim, o cálculo é feito da última parada para a primeira: o minuto final fica reservado para a Produção, a última parada termina no máximo um minuto antes do corte e cada parada anterior termina antes da próxima já encaixada. Depois do encaixe, os itens são gravados em ordem cronológica. Quando não há um minuto válido para uma parada, ela não é criada; se não houver minuto válido para a Produção, o pacote é descartado.

O pacote possui `status=0` para pendente, `status=1` para integrado, `status=2` para erro e `status=3` para processando. O log e a data/hora registram o último retorno. A tela `/producao/log-tempos-erp/` pagina 20 pacotes, expande para mostrar seus itens e permite envio individual ou dos pendentes. O botão global é bloqueado enquanto a fila estiver processando; também não deixa enviar um pacote quando houver pacote anterior pendente, com erro ou processando para a mesma chave de OP. Superusuários podem excluir pacotes ainda não integrados e, dentro de um pacote pendente ou com erro, excluir individualmente somente itens do tipo Parada; itens de Produção e pacotes integrados ou em processamento não podem ser excluídos.

O worker `fila_tempos_erp` reserva os pacotes no banco antes de enviar, usa lock em memória e processa apenas uma chave de OP por vez. O envio usa SOAP no serviço `ApontamentoTempos`, imprime o envelope enviado e a resposta bruta no processo, grava o retorno no pacote e respeita timeout de 180 segundos. O `envia_pendencias` dispara a fila automaticamente para pacotes pendentes ou com erro; pacotes que permanecerem em processamento além do timeout voltam a pendente

quando o lock da fila estiver livre.

Quando a tela de logs ou apontamento precisa aumentar uma quantidade já integrada, o SIGMA usa `AumentarApontamento`. O motivo não é criar uma nova operação desconectada, mas complementar o apontamento existente no ERP. A regra Senior evita duplicidade, incrementa a quantidade e gera as pendências proporcionais de componentes da mesma forma controlada do apontamento principal.

Fluxo resumido:

1. Usuário seleciona OP/recurso na tela.
2. Sistema identifica o recurso e escolhe a tela de apontamento conforme o `view_id` configurado no cadastro do recurso.
3. Sistema valida operador no ERP.
4. Sistema grava um ou mais registros locais em `producao.apontamento`.
5. Na v1, se o registro local foi criado com sucesso, a validação do operador é limpa para o próximo apontamento.
6. Registro inicia como `status=0`.
7. Worker `fila_logs_apontamentos` reserva para `status=2`.
8. Envia `Apontamentos` com `wacao=APONTAR-OP` para efetivar a produção no ERP e gerar o controle de baixa posterior dos componentes.
9. Interpreta `<waRetorno>`.
10. Se sucesso, marca `status=1`.
11. Se erro, volta para `status=0` com log.

Payload principal:

- `empresa`.
- `CodOri`.
- `NumOrp`.
- `NumCad`.
- `CodEtg`.
- `SeqRot`.
- `QtdRel`.
- `QtdRfg`.
- `DatMov`.
- `HorMov`.
- `NumBob`.
- `NumMaq`.
- `CodLot`.

No apontamento normal com quantidade produzida ou refugo, o lote não é digitado livremente pelo operador. O SIGMA gera o `CodLot` a partir do `loteatual` da empresa, grava esse lote no registro local e incrementa o próximo lote disponível. Quando a OP gera mais de uma bobina, o sistema divide a quantidade entre as bobinas e gera um lote para cada registro criado.

O `CodLot` só deixa de ser enviado quando o registro local fica sem lote, situação usada para apontamento sem quantidade produzida e sem refugo. Nesse caso, o JSON enviado ao Senior não recebe a chave `CodLot`; na regra LSP o valor fica vazio e o apontamento segue sem criação de novo lote de produto acabado.

8.4 Apontamento de componente

O apontamento de componente registra a baixa ou consumo de componentes vinculados a uma OP. Esse fluxo tem uma particularidade importante: o valor informado pelo operador pode ser um lote ERP ou um paleta WMS. Por isso, antes de chamar o webservice do ERP, o SIGMA resolve qual é a origem real do dado.

Na tela `apontamentos_v2`, o valor informado para o lote/paleta do componente precisa ter pelo menos 4 caracteres. Valores menores são bloqueados antes de gravar `ApontamentoComponente`, para evitar registros acidentais de leituras incompletas.

Quando o valor é um paleta WMS, o sistema consulta a tabela de paletes importados para descobrir o lote ERP, o componente, a derivacao e a quantidade disponível. Quando não encontra o paleta, tenta tratar o valor como lote ERP e consulta o saldo no estoque. Essa ordem evita baixar componente usando o identificador errado.

Depois que o ERP confirma o apontamento do componente, se o registro veio de paleta WMS, o SIGMA cria um ajuste de estoque no WMS para zerar ou ajustar o paleta consumido. Essa etapa acontece somente depois do sucesso no ERP, para não alterar o WMS se o ERP não aceitou o movimento.

O envio ao webservice `ApontamentoComponente` existe porque o componente informado pela operação precisa entrar no controle do ERP como consumo vinculado a OP. A regra Senior valida o saldo do lote do componente, calcula a quantidade

liquida da OP quando aplicável, aponta a OP e grava o componente recebido como pendência de baixa em `USU_TBXACMP`. Esse comportamento é importante porque o componente informado pode não ser exatamente o mesmo que o ERP baixaria automaticamente pela estrutura original da OP.

A baixa efetiva do componente não acontece diretamente nesse mesmo fluxo. O apontamento cria a pendência com `USU_SITPEN=1` e `USU_IDEUNI`; depois a regra orquestradora 230 reserva a pendência, muda para `USU_SITPEN=2` e envia o registro de forma assíncrona para o webservice interno `BaixaComponenteERP`. O processamento final marca a pendência como sucesso (3) ou erro (4).

Fluxo:

1. Usuário informa/bipa etiqueta.
2. Sistema grava `ApontamentoComponente`.
3. Worker consulta `USU_TPALWMS` por empresa e paleta.
4. Se encontrar:
 - Usa lote ERP vinculado.
 - Usa componente, derivacao e quantidade disponível do paleta.
 - Guarda `PalWms` para ajuste WMS posterior.
5. Se não encontrar na `USU_TPALWMS`, consulta `E210DLS` por lote pois pode ser uma etiqueta de bobina refugada gerada pelo SIGMA.
6. Envia `ApontamentoComponente` com `wacao=APONTAR-COMPONENTE` para registrar o consumo do componente no controle do ERP e gerar a pendência em `USU_TBXACMP`.
7. Se ERP OK e havia paleta WMS:
 - Cria `WMS_IntegraçãoOP`.
 - `tipo_envio=ajuste`.
 - `quantidade=0`.
 - paleta igual ao paleta bipado.
 - Envia para WMS `ajuste_estoque`.
8. Regra orquestradora 230 busca pendências com `USU_SITPEN=1`, reserva como `USU_SITPEN=2` e envia para `BaixaComponenteERP`.
9. `BaixarComponentes` processa a baixa pelo `USU_IDEUNI` e atualiza `USU_SITPEN` para 3 em sucesso ou 4 em erro.
10. Se WMS falhar na criação da pendência, componente volta para pendente com log.

8.4.1 Painei Movimentar Componentes

O painel **Movimentar Componentes** é somente consulta e usa a origem de OP 110 como filtro. O acesso é permitido para staff ou para quem possui a permissão `Pode visualizar componentes movimentar`; usuário não staff consulta apenas a empresa da sua filial. O filtro **Máquina** lista somente recursos com OP aberta nessa origem, em situação A ou L e marcada para movimentação (`MOVORP='S'`); ao selecionar o recurso, a tela é recarregada com suas OPs. Componentes das origens 230, 405 e 410 são exibidos como **Bobina**. O alerta de consumo começa em 90% e torna-se crítico acima de 100%.

Uma OP é atual quando possui início e ainda não possui fim; as demais aparecem como próximas. OP concluída pelo realizado não é exibida. Quando houver mais de uma OP atual, o painel também consulta o último apontamento: uma OP sem movimentação há mais de quatro horas é removida somente se existir outra OP atual com movimentação mais recente.

O painel permite consultar os lotes já baixados do componente na OP pela transação 90251 e, para componentes classificados como bobina, consultar no WMS os paletes/lotos disponíveis para cobrir apenas o saldo ainda necessário da OP.

8.5 Estorno de componente

O estorno de componente é o fluxo de reversão de baixa de componente. Ele não é criado em qualquer ajuste de apontamento. O SIGMA só gera registros de estorno quando o acerto do apontamento é para menor, ou seja, quando a nova quantidade do lote fica abaixo da quantidade já registrada. Para cada sequência afetada, o SIGMA envia `DiminuirApontamento` com `wacao=DIMINUIR-OP`. A regra personalizada chama internamente o serviço nativo `Acertar` do ERP com a nova quantidade final; somente com retorno de sucesso ela zera a sequência inicial relacionada quando aplicável.

Quando o ajuste é para maior, o comportamento é diferente: não há estorno. O SIGMA faz um apontamento complementar pelo fluxo de aumento de apontamento, enviando apenas a diferença positiva para o ERP. Assim, aumento de quantidade e redução de quantidade não usam a mesma regra: aumento complementa o apontamento; redução acerta a quantidade e gera estorno dos componentes correspondentes.

O retorno do acerto do apontamento acontece de forma síncrona para a tela. No aumento, a tela aguarda o retorno do

webservice de apontamento complementar. Na redução, a tela aguarda o retorno de `DiminuirApontamento`. Apenas depois de um retorno OK na redução o SIGMA grava os componentes que precisam ser estornados. O envio desses estornos ao `EstornaComp` é a parte assíncrona: os registros ficam em fila local e são processados pelo worker de estorno de componentes.

Na exclusão de lote integrado, a última redução do rateio também recebe `ExcluirLote=S`. Depois do `Acertar` e do ajuste da sequência anterior, a mesma execução marca o lote como excluído em `E210DLS.USU_SITLOT='E'`, usando empresa, lote e depósito de consulta. Não há uma segunda chamada ao webservice para essa marcação. A partir daí, o SIGMA grava o histórico local de exclusão com `status=3`.

O envio ao `EstornaComp` existe para devolver o componente ao estado correto no ERP. A regra Senior localiza o movimento anterior de consumo, prepara a OP com o lote e depósito usados na baixa original e chama a rotina de estorno de componentes. Isso evita estornar apenas uma quantidade solta, sem contexto de OP, componente, lote e depósito.

Se o ERP aceita o estorno, o registro local passa para integrado. Se o ERP rejeita ou ocorre erro de comunicação, o estorno volta a ficar pendente com o motivo registrado no log.

Fluxo:

1. Usuário solicita acerto de apontamento.
2. Se a nova quantidade for maior, o SIGMA envia apontamento complementar, aguarda o retorno na tela e não cria estorno.
3. Se a nova quantidade for menor, o SIGMA rateia a redução pelas sequências do lote e chama `DiminuirApontamento`; a regra executa o `Acertar` nativo e o ajuste complementar da sequência inicial.
4. Se o acerto retornar OK, o SIGMA calcula os componentes afetados pela redução e grava registros em `producao.estorno_comp`.
5. Worker reserva os estornos pendentes.
6. Envia `EstornaComp` para desfazer a baixa do componente no ERP.
7. Se retorno OK, marca integrado.
8. Se erro, volta para pendente.

8.6 Liberação de lotes

A liberação de lotes é o fluxo da qualidade para retirar um lote recém produzido da condição de pendência/consulta e movimentá-lo para o depósito/local de armazenamento aprovado. O SIGMA não trabalha apenas com uma decisão visual: ele valida a situação no ERP, grava um registro local de liberação, gera movimento para o ERP e cria a pendência correspondente para o WMS.

O acesso a essa tela é controlado pela permissão **Pode acessar a tela de liberação de lotes**. A única ação de destinação nessa tela é **Liberar**, que exige a permissão **Pode destinar lotes na tela de liberação**.

Os depósitos de origem, depósito destino ERP, local WMS e transação de movimento não são fixos no código. Eles são resolvidos a partir dos parâmetros da filial e, quando houver configuração mais específica, do centro de recurso. Isso permite que recursos ou centros tenham regras diferentes sem alterar o programa.

O fluxo também consulta Alchemy para trazer informação de análise da bobina, ajudando a qualidade a decidir antes de liberar.

Na liberação direta, o envio ao ERP usa `MovimentarEstoque` com `acaoBotao` igual a A. O motivo do envio é mover o saldo para o depósito aprovado e gravar a situação customizada do lote como liberada. Nesse caso, o SIGMA também cria pendência WMS do tipo `novo_lote`, porque o lote liberado precisa ser recebido/criado no WMS no local de armazenamento definido.

Enviar um lote para Área Vermelha é a etapa de triagem física disponível nesta tela para origens permitidas: usa `MovimentarEstoque`, a transação interna padrão da filial e o depósito de Área Vermelha do recurso ou filial, gravando situação V. A análise, a decisão de liberar/refugar/reclassificar e as integrações WMS continuam exclusivamente na reunião de **Área Vermelha**.

Refugo e reclassificação não são tratados como liberação direta. Eles acontecem depois da avaliação da Área Vermelha, quando a reunião define o destino do lote. Nessa etapa, se houver refugo ou reclassificação, o envio ao ERP usa `TransferenciaProduto`, porque o saldo precisa ser transformado para outro produto/derivação/lote. Se a decisão final for liberar quantidade sem troca de produto, o envio usa `MovimentarEstoque`.

Fluxo:

1. Tela consulta lotes no ERP com saldo em depósitos configurados.
2. Busca dados em `E210DLS`, `E900EQ`, `E725CRE`, `E075PR0`.
3. Consulta Alchemy para flag de análise.
4. Usuário escolhe liberar.

5. Sistema valida `USU_SITLOT` no ERP.
6. Sistema resolve parâmetros:
 - Depósito destino ERP.
 - Transação ERP.
 - Local WMS.
7. Para liberação direta, grava um registro local em `qualidade.liberacao_lote`.
8. A partir desse registro, grava uma pendência local em `qualidade.wms_integracao_op` com tipo `novo_lote`.
9. Dispara a fila WMS para enviar o lote ao WMS no local de armazenamento definido.
10. Na sequência, reserva o registro de `qualidade.liberacao_lote` e dispara a fila de consulta de lotes para enviar `MovimentarEstoque` ao ERP.

Na liberação direta, o código dispara a fila WMS antes de disparar a fila ERP. Isso não significa que os dois sistemas sejam atualizados em uma mesma transação. O SIGMA cria registros locais e aciona duas integrações assíncronas: uma para o WMS e outra para o ERP. Como são filas separadas, cada uma pode concluir, falhar ou ser reprocessada de forma independente.

8.7 Área vermelha

A área vermelha é o fluxo de avaliação de lotes que exigem decisão de qualidade antes da destinação final. O processo funciona por reunião: os participantes avaliam os lotes, definem quantidades para liberar, refugar ou reclassificar, registram motivos e observações, e só depois o sistema envia os movimentos.

Esse desenho evita que uma decisão parcial seja enviada ao ERP ou ao WMS antes do fechamento da análise. Enquanto a reunião está aberta, os registros ERP (`LiberacaoLote`) e as pendências WMS vinculados a ela ficam bloqueados para envio. O envio só é liberado depois do fechamento da reunião, quando o sistema grava a data de fechamento, atualiza a data de geração dos registros e reserva os lotes pendentes para processamento.

O módulo também faz uma resolução mais complexa da origem do lote. Ele consulta movimentos do ERP para encontrar OP, origem e recurso reais, inclusive quando o lote atual é resultado de transformações ou transferências anteriores. Para o WMS, a consulta via DBLINK identifica se o par SKU/lote já existe. Quando já existe, o fluxo usa o local encontrado no WMS; somente quando o par não existe usa o local padrão de Área Vermelha configurado no recurso ou, na falta dele, na filial. Sem um local resolvido, o salvamento é cancelado. Liberar gera ajuste; refugo e reclassificação geram `novo_lote`. Quando o lote original já existe no WMS e a decisão não contém uma linha de liberar, o sistema também cria o ajuste de quantidade zero para manter o lote original consistente.

Os motivos exibidos na reunião vêm do cadastro de defeitos/motivos do ERP, na tela `F011DEF`. O SIGMA lista apenas motivos ativos e marcados com `Utiliza na Área Vermelha = S`, campo customizado `USU_UTIAVE` na tabela `E011DEF`. Isso evita que a tela permita escolher motivos que existem no ERP, mas não devem ser usados nesse processo de qualidade.

O atalho **Área Vermelha** em Liberação de Lotes move somente o saldo para o depósito de triagem e grava a situação V; não define destinação final nem cria liberação, refugo ou reclassificação. Essas decisões permanecem no contexto da reunião, que só libera suas filas após o fechamento.

Quando a decisão da reunião gera refugo ou reclassificação, o envio ao ERP usa `TransferenciaProduto`. Nessa regra, o motivo escolhido também define como o centro de custo será atribuído ao movimento de estoque. O campo `Forma de Atribuição de Centro de Custo` (`USU_ATRCCU`) pode assumir três comportamentos: `OF`, usando a OP de fabricação do lote; `OC`, usando a OP que consumiu o lote; ou `CF`, usando um centro de custo fixo. Quando a forma é `CF`, o ERP usa o campo `Centro de Custo` (`USU_CCUFIX`) cadastrado no próprio motivo.

Na regra atual do webservice, o comportamento é:

- `OF`: busca em `E900EQQ` a primeira operação vinculada ao próprio lote classificado (`CODLOT`) que tenha centro de recurso preenchido. Com esse centro de recurso, consulta `E725CRE.CODCCU` e usa esse centro de custo no movimento. A regra atual não busca primeiro o movimento de entrada em `E210MVP` para descobrir a OP de fabricação; ela resolve diretamente pela operação registrada para o lote.
- `OC`: busca em `E210MVP` o último movimento do lote com transação `90251`. A origem e o número do documento desse movimento indicam a OP de consumo. Com essa OP, busca a primeira operação em `E900EQQ`, obtém o centro de recurso e depois consulta `E725CRE.CODCCU`.
- `CF`: usa diretamente o centro de custo fixo cadastrado no motivo (`USU_CCUFIX`).

O centro de custo encontrado é aplicado tanto no movimento de saída quanto no movimento de entrada da transferência entre produtos. Se a regra configurada não conseguir resolver um centro de custo, o webservice não executa a transferência e retorna erro.

Fluxo:

1. Usuário abre reunião ou usa reunião aberta.
2. Sistema consulta lotes elegíveis no ERP.
3. Filtros principais:
 - Empresa do usuário.
 - Depósitos de área vermelha.
 - Origens permitidas pela filial.
 - Produto com `USU_CONREL='S'`.
 - Saldo maior que zero.
4. As origens permitidas vêm do parâmetro `origens_area_vermelha` da filial. O select principal aplica esse parâmetro sobre `E075PRO.CODORI`, então a tela só lista produtos cuja origem esteja liberada para avaliação de Área Vermelha naquela filial. O lote no depósito de Área Vermelha não precisa estar marcado como consumo real (`USU_CONREL='S'`).
5. Sistema busca referência real de movimento em `E210MVP` e cadeia de lotes.
6. Sistema busca recurso/OP em `E900EQ`.
7. Sistema consulta o WMS via DBLINK para verificar se o par SKU/lote já existe e qual é o local atual do estoque.
8. A consulta identifica se o par SKU/lote já existe no WMS. Para gerar as pendências, usa o local encontrado quando existir; somente para lote ainda inexistente usa o local padrão de Área Vermelha do recurso ou, na falta dele, da filial. Sem local resolvido, cancela o salvamento:
 - Liberar gera ajuste.
 - Refugar e reclassificar geram `novo_lote`.
 - Se o lote original existe no WMS e não há uma linha de liberar, gera também `ajuste` com quantidade zero para o lote original.
9. Sistema consulta Alchemy para análise/observação.
10. Sistema carrega os motivos ativos do ERP permitidos para Área Vermelha:
 - Cadastro `F011DEF`.
 - Motivo ativo.
 - Campo `Utiliza na Área Vermelha` igual a `S`.
12. Usuário destina quantidades:
 - Liberar.
 - Refugar.
 - Reclassificar.
13. Para refugo e reclassificação, o motivo escolhido define a forma de atribuição do centro de custo no movimento ERP:
 - `OF`: centro de custo pela primeira operação registrada para o lote em `E900EQ`.
 - `OC`: centro de custo pela última OP que consumiu o lote, encontrada em `E210MVP` pela transação `90251`.
 - `CF`: centro de custo fixo cadastrado no motivo.
14. Sistema cria linhas `LiberacaoLote`.
15. Sistema cria pendências WMS conforme destino.
16. Enquanto a reunião está aberta, os envios ERP e WMS ficam bloqueados.
17. Após o fechamento, as filas podem enviar ERP e WMS.

8.8 Consulta de lote, etiqueta e rastreamento

A consulta de lote é a tela de acompanhamento das destinações já registradas pela qualidade. Ela permite ver o status das liberações, acompanhar logs de envio, reenviar registros pendentes, excluir o que ainda não foi integrado e imprimir etiquetas quando o movimento já foi concluído.

Antes da impressão, o sistema consulta saldo no ERP para confirmar se o lote ainda possui quantidade suficiente no depósito destino. Essa validação evita imprimir etiqueta para um lote que já não tem saldo compatível.

O rastreamento monta uma linha do tempo do lote a partir dos movimentos do ERP, incluindo transformações, transferências, origem, destino e informações de qualidade. Quando há vínculo com Alchemy, também busca dados de análise por bobina/máquina.

Comportamentos:

- Lista registros `LiberacaoLote`.
- Agrupa por lote/produto/derivacao/bobina.

- Permite envio manual de registros pendentes.
- Valida saldo no ERP antes de imprimir etiqueta.
- Gera etiqueta com código de barras Code128 e QR Code de rastreamento.
- Rastreamento consulta movimentos do ERP e dados de qualidade/Alchemy.

8.9 Status de recursos

O painel de status de recursos consolida a situação operacional dos recursos produtivos. Ele cruza informações locais e do ERP para mostrar OP ativa, operador, produção do dia e produção desde a última troca de OP.

As paradas de máquina são controladas em `ParadaMaquina`, onde `fim` vazio indica parada aberta. Cada parada pertence fisicamente a um recurso e se associa a um ou mais períodos de `LogTrocaOPAtiva` desse recurso, sem repetir os campos da OP. O motivo não fica na parada: cada motivo é registrado em `JustificativaParada`, permitindo dividir a duração em várias sequências. A parada registra início, fim, código do operador, usuário que alterou por último, tipo manual/sinal e `data_hora`. Uma parada só deixa de bloquear o apontamento quando está fechada e o tempo de suas justificativas cobre toda a duração; o fim pode ter sido informado pelo operador ou pelo processo automático. A troca normal de OP é bloqueada enquanto existir parada aberta ou pendente; se uma parada física acompanhar uma troca autorizada, ela permanece contínua e cada período recebe somente sua interseção temporal. A produção realizada é consultada no ERP, porque o ERP é a fonte oficial dos apontamentos já efetivados.

Comportamentos:

- Lista recursos por empresa/centro.
- Mostra OP ativa aberta.
- Busca operador no ERP.
- Busca produção total do dia em `E900E0Q`.
- Busca produção desde última troca de OP.
- Usa dados locais de OP ativa e paradas de máquina.

8.10 Manutenção

O módulo de manutenção controla chamados e ordens de serviço relacionados aos recursos cadastrados. Todo acesso exige autenticação: `Pode acessar Chamados` libera a aba, a listagem e a abertura de chamados; `Pode acessar OS` libera a aba de ordens de serviço. `Pode listar todos os chamados` permite consultar qualquer chamado, inclusive os que o usuário não abriu, não observa e não atende. `Pode manipular chamados` permite editar e excluir chamados. O usuário logado que abre o chamado é registrado como autor da primeira interação. O QR Code por recurso direciona para a abertura já vinculada ao equipamento correto, exigindo login antes do acesso.

Chamados registram categoria, prioridade, status, recurso, responsáveis, observadores e interações. Ordens de serviço podem nascer de chamados ou ser abertas diretamente, com responsáveis, previsão, execução real e histórico de interações.

O módulo também envia notificações por e-mail para envolvidos, usando o SMTP configurado no sistema.

Chamados:

1. Usuário abre chamado por tela ou QR Code de recurso.
2. Sistema grava chamado e interação inicial.
3. Lista chamados conforme permissão:
 - Staff ou `Pode listar todos os chamados` ve todos.
 - Usuário com `Pode acessar Chamados`, sem a permissão global, ve chamados que abriu, observa ou atende.
4. Envia e-mail para envolvidos quando aplicável.

Ordens de serviço:

1. Usuário autorizado abre OS.
2. OS pode nascer de um chamado.
3. Responsáveis são armazenados como lista de IDs.
4. Interações registram execução, início/fim e descrição.
5. E-mails são enviados em eventos relevantes.

9. Parametrizações críticas

Os parâmetros são a parte configurável da regra de negócio. Eles evitam que depósitos, locais WMS, transações ERP e limites fiquem fixos no código. Isso é importante porque a mesma regra geral pode precisar se comportar de forma diferente por filial, centro de recurso ou recurso.

A regra de leitura é hierárquica:

1. O sistema parte dos parâmetros da filial.
2. Se o centro de recurso tiver um valor preenchido, esse valor substitui o da filial.
3. Para alguns campos específicos, o recurso pode ter configuração própria.

Exemplo prático: uma filial pode ter um depósito padrão de liberação, mas determinado centro pode liberar para outro depósito. Nesse caso, o usuário não precisa escolher manualmente a cada operação; o SIGMA resolve o parâmetro efetivo conforme o recurso/centro envolvido.

Isso torna os parâmetros uma das partes mais críticas do sistema. Um parâmetro incorreto pode fazer uma consulta buscar saldo no depósito errado, uma liberação criar pendência WMS para local errado ou uma movimentação ERP usar transação inadequada. Por esse motivo, alterações nesses cadastros devem ser tratadas como mudança operacional, não apenas manutenção de cadastro.

Parâmetro	Nível	Impacto
tempo_sem_comunicacao_manual	Filial/Recurso	Tempo para apontamento manual.
limite_apontamento_minimo	Filial/Recurso	Limite mínimo de apontamento.
limite_apontamento_maximo	Filial/Recurso	Limite máximo de apontamento.
deposito_apontamento_erp	Filial/Centro	Depósitos consultados para lotes apontados.
deposito_armazenamento_erp	Filial/Centro	Depósito destino de liberação.
deposito_armazenamento_wms	Filial/Centro	Local WMS de liberação.
deposito_area_vermelha_erp	Filial/Centro	Depósito de área vermelha.
deposito_area_vermelha_wms	Filial/Centro	Local WMS de área vermelha.
codtns	Filial	Transação ERP de transferência.
codtns_area_vermelha	Filial	Transação ERP específica de área vermelha.
produto_refugo	Filial/Centro	Produto destino de refugo.
derivacao_refugo	Filial/Centro	Derivação destino de refugo.
origens_area_vermelha	Filial	Origens elegíveis na área vermelha.
transacoes_saida_consumo_producao	Filial	Ajuda rastreamento/referência de movimento.
transacoes_entrada_producao_consumo	Filial	Ajuda rastreamento/referência de movimento.

Parâmetro	Nível	Impacto
<code>cod_alchemy</code>	Centro	Vínculo do recurso com máquina Alchemy.

Descrição dos principais grupos de parâmetros:

Parâmetros de apontamento controlam limites mínimos e máximos de quantidade e regras de apontamento manual. Eles protegem a operação contra apontamentos fora de faixa e ajudam a separar apontamento automático, apontamento manual e situações sem comunicação.

Parâmetros de depósito ERP definem onde o sistema consulta saldo e para onde movimenta lote no ERP. Eles aparecem em liberação de lote, área vermelha, consulta de lote e validações de estoque. Se o depósito configurado não corresponder ao processo real, o sistema pode concluir que não existe saldo ou direcionar movimento para local indevido.

Parâmetros de local WMS definem o endereço usado ao criar pendências para o WMS. Eles são diferentes dos depósitos ERP porque representam o lado logístico/WMS do processo. Um lote pode ter movimento no ERP e, ao mesmo tempo, precisar ser recebido ou ajustado em local WMS específico.

Parâmetros de área vermelha definem quais origens entram na tela, quais depósitos são considerados e quais transações ajudam a rastrear a referência real do lote. Essa configuração é sensível porque a área vermelha trata material em exceção, onde a decisão pode ser liberar, refugar, reclassificar ou manter pendente de reunião.

Parâmetros de refugo definem produto e derivacão usados quando a qualidade destina quantidade como refugo. Isso evita que o usuário tenha que informar manualmente o destino técnico do refugo em cada processo.

O parâmetro Alchemy liga um centro ou recurso do SIGMA a uma máquina no Alchemy para buscar análises. Ele é usado para complementar a decisão de qualidade e manter informações de bobina alinhadas ao recurso correto.

10. Segurança e acessos

O SIGMA tem dois níveis principais de segurança: autenticação e autorização. Autenticação responde “quem é o usuário?”. Autorização responde “o que esse usuário pode fazer?”.

10.1 Autenticação

O login usa LDAP/Active Directory como primeira opção e o backend Django como apoio. O uso de LDAPS com certificado CA protege a comunicação com o controlador de domínio.

Quando um usuário permitido entra pela primeira vez, o SIGMA pode criar o cadastro local automaticamente. Esse cadastro local é necessário para vincular filial, operador ERP, permissões e histórico de ações.

10.2 Autorização

Depois do login, as telas e ações usam permissões. Algumas telas exigem apenas usuário autenticado; outras exigem permissão específica de produção, qualidade ou manutenção. Ações de maior risco, como excluir filas pendentes ou registros não integrados, são restritas a superusuário ou staff conforme o caso.

Na tela de apontamento, a permissão **Pode apontar** libera a validação de operador, a troca de operador, a troca de OP ativa e o apontamento. O desacoplamento da OP ativa exige que o operador tenha sido validado na tela.

Essa separação impede que um usuário operacional comum acesse manutenção administrativa, cadastros sensíveis ou operações de reprocessamento indevidas.

No Sequenciamento, o acesso à tela e ao menu exige **Pode acessar Sequenciamento**; consolidar ou executar o sequenciamento automático exige **Pode consolidar Sequenciamento ERP**. Essas são as únicas permissões do módulo exibidas na administração de usuários e grupos.

No PCP, o Calendário de OPs exige a permissão **Pode visualizar Calendário de OPs** (`pcp.pode_visualizar_calendario_ops`), atribuível diretamente ao usuário ou por grupo. Usuários staff também possuem acesso.

10.3 Observações de segurança

Pontos de segurança:

- Em produção, o sistema aceita somente o host `sigma.indaialpapel.com.br`.
- O Nginx redireciona o domínio de HTTP para HTTPS e recusa acesso direto pelo IP.

- O Nginx não informa sua versão nem o sistema operacional nos headers HTTP.
- Cookies de sessão e CSRF são enviados somente por HTTPS.
- As senhas ficam em variáveis de ambiente, o que é adequado.
- Este documento não registra senhas nem usuários técnicos.

O HTTPS termina no Nginx, que encaminha X-Forwarded-Proto ao Django. A aplicação reconhece a requisição original como segura por SECURE_PROXY_SSL_HEADER.

10.3.1 HTTPS e renovação do certificado

O Nginx publica o SIGMA somente em `https://sigma.indaialpapel.com.br`. O domínio em HTTP redireciona para HTTPS; requisições pelo IP são recusadas. O Daphne escuta apenas em `127.0.0.1:8000`, sem exposição direta da aplicação na rede.

O domínio resolve no DNS interno para `172.16.30.63` e não possui resolução nos servidores DNS públicos. O firewall do servidor aceita HTTP e HTTPS somente quando o endereço de origem pertence à faixa corporativa `172.16.0.0/16`; as demais origens são bloqueadas. O acesso pela VPN corporativa depende de o tráfego chegar ao servidor por essa faixa autorizada.

O certificado em uso é coringa para `*.indaialpapel.com.br`, emitido pela Let's Encrypt, com validade até 10/09/2026. Os arquivos ativos ficam em:

```
/etc/ssl/sigma/cert.pem
/etc/ssl/sigma/chain.pem
/etc/ssl/sigma/fullchain.pem
/etc/ssl/sigma/privkey.pem
```

O Nginx usa `fullchain.pem` em `ssl_certificate` e `privkey.pem` em `ssl_certificate_key`. O PFX recebido da infraestrutura é usado somente durante a conversão e não permanece no servidor depois da instalação.

Para consultar o certificado publicado e sua validade:

```
openssl s_client \
  -connect sigma.indaialpapel.com.br:443 \
  -servername sigma.indaialpapel.com.br </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Antes do vencimento, solicitar à infraestrutura um novo PFX coringa e sua senha. Enviar o arquivo temporariamente para `/home/nexus/certificado_sigma.pfx` e executar:

```
install -d -m 700 /tmp/sigma-cert
read -s -p "Senha do PFX: " PFX_PASSWORD
export PFX_PASSWORD

openssl pkcs12 \
  -in /home/nexus/certificado_sigma.pfx \
  -passin env:PFX_PASSWORD -clcerts -nokeys 2>/dev/null \
  | openssl x509 -out /tmp/sigma-cert/cert.pem

openssl pkcs12 \
  -in /home/nexus/certificado_sigma.pfx \
  -passin env:PFX_PASSWORD -cacerts -nokeys 2>/dev/null \
  | awk '/-----BEGIN CERTIFICATE-----/{p=1} p{print} /-----END CERTIFICATE-----/{p=0}' \
  > /tmp/sigma-cert/chain.pem

openssl pkcs12 \
  -in /home/nexus/certificado_sigma.pfx \
  -passin env:PFX_PASSWORD -nocerts -nodes 2>/dev/null \
  | openssl pkey -out /tmp/sigma-cert/privkey.pem

unset PFX_PASSWORD
cat /tmp/sigma-cert/cert.pem /tmp/sigma-cert/chain.pem \
  > /tmp/sigma-cert/fullchain.pem
```



```
openssl x509 -in /tmp/sigma-cert/cert.pem \
  -noout -subject -issuer -dates -ext subjectAltName
openssl verify -untrusted /tmp/sigma-cert/chain.pem \
  /tmp/sigma-cert/cert.pem

sudo install -o root -g root -m 644 \
  /tmp/sigma-cert/cert.pem \
  /tmp/sigma-cert/chain.pem \
  /tmp/sigma-cert/fullchain.pem \
  /etc/ssl/sigma/
sudo install -o root -g root -m 600 \
  /tmp/sigma-cert/privkey.pem \
  /etc/ssl/sigma/privkey.pem

sudo nginx -t && sudo systemctl reload nginx
rm -f /home/nexus/certificado_sigma.pfx
rm -rf /tmp/sigma-cert
```

Depois da recarga, executar novamente a consulta do certificado publicado e confirmar o novo período de validade e o nome *.indaialpapel.com.br.